

Beyond Confidence: Stability-Aware Test-Time Adaptation for LLM Reasoning

Bincheng Gu¹, Min Gao^{1*}, Zongwei Wang¹, Yibing Bai¹, Yulan He², Junliang Yu³

¹Key Laboratory of Dependable Service Computing in Cyber Physical Society, Chongqing University, China

²King's College London, London, UK

³Griffith University, Brisbane, Australia
gaomin@cqu.edu.cn,

Abstract

Test-time adaptation has emerged as a lightweight alternative to costly post-training for improving the reasoning capabilities of Large Language Models (LLMs) on downstream tasks. Predictive entropy provides a model-derived signal for such adaptation, guiding models toward higher-confidence reasoning states without external verifiers or reward models. However, higher confidence does not necessarily imply correctness, as LLMs may remain highly confident along incorrect reasoning trajectories. We observe that high-confidence reasoning is more likely to be correct when confidence remains stable under local perturbations. Based on this observation, we propose Test-Time Adaptation via Stability-Aware Confidence Optimization (TASCO), a framework that incorporates local stability into confidence-based test-time adaptation while keeping the LLM frozen. TASCO operationalizes local stability by optimizing a lightweight task-level prefix under two alternative perturbation strategies: Random Perturbation promotes distributional stability across trajectories induced by nearby perturbed prefixes, whereas Sharpness-Aware Perturbation targets worst-case local sensitivity. Experiments demonstrate that TASCO improves reasoning accuracy and token efficiency across diverse LLMs and reasoning benchmarks, while behavioral analyses show that it maintains stable confidence under local perturbations without prematurely concentrating the model's predictive distribution.

Introduction

Large Language Models (LLMs) have demonstrated strong reasoning capabilities on complex tasks such as mathematical reasoning and code generation (Guo et al. 2025; Jaech et al. 2024). Traditionally, adapting these models to downstream tasks relies on post-training methods that update model parameters, such as supervised fine-tuning or reinforcement learning (Shao et al. 2024; Lambert et al. 2024). Although effective, these methods require substantial computational resources, motivating recent work to keep model parameters frozen and instead optimize continuous inputs or internal representations at inference time (Hu et al. 2025b).

One effective paradigm is to optimize a small set of continuous variables, including soft prompts, steering vectors, and latent representations, to guide a frozen LLM toward task-specific reasoning states (Xu et al. 2025b; Li et al. 2025;

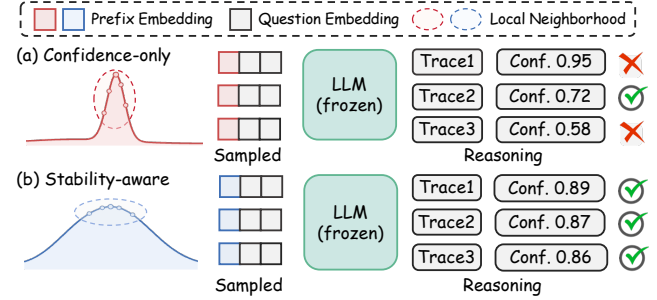


Figure 1: Comparison of confidence-only and stability-aware optimization under prefix perturbations across trajectories yielding correct and incorrect answers.

Sinii et al. 2025). These approaches differ mainly in the source of their optimization signals. Some rely on external verifiers or reward models to score answers or reasoning trajectories (Khalifa et al. 2025; Nguyen and Le 2026), whereas verifier-free methods use the model's output entropy as an intrinsic confidence signal (Ye, Liang, and Shan 2025).

However, entropy-based test-time optimization assumes that an LLM is more likely to answer correctly when its predictive entropy is lower during reasoning (Agarwal et al. 2026). This assumption can fail on complex reasoning tasks, where LLMs can be poorly calibrated and remain highly confident along incorrect reasoning trajectories (Jurayj, Cheng, and Van Durme 2025; Huang et al. 2025). When confidence is the only test-time objective, entropy minimization cannot distinguish high confidence associated with reliable reasoning from spurious confidence along an incorrect trajectory, potentially steering the model toward a locally fragile high-confidence state (Chen et al. 2026a; Zhao et al. 2026).

How can we guide an LLM toward a high-confidence state that supports reliable reasoning? Inspired by the connection between flat minima and generalization (Keskar et al. 2016), we examine the local stability of high-confidence states. Specifically, high confidence that remains stable under local perturbations to the steering variable is more likely to stem from reliable reasoning than from a fragile overconfident state (Fig. 1). We measure this stability using trajectory-level confidence variance. As shown in Fig. 2, the low-variance group consistently achieves higher answer accuracy than

*Corresponding author.

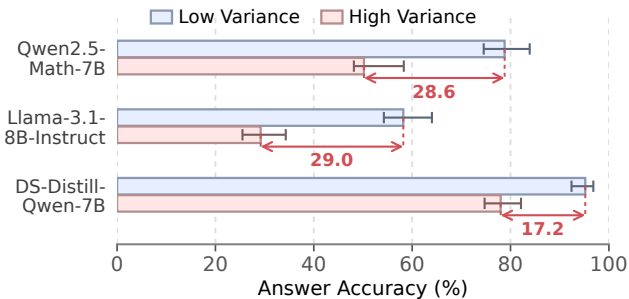


Figure 2: Answer accuracy for low- and high-variance query groups defined by the median trajectory-confidence variance.

the high-variance group across representative models from three families. Further analysis shows that the low-variance group remains more accurate among queries with comparable mean confidence (see Appendix B for details). Together, these results suggest that local stability complements confidence magnitude in characterizing reasoning reliability.

Guided by this observation, we propose Test-Time Adaptation via Stability-Aware Confidence Optimization (TASCO), a label-free framework for guiding frozen LLMs toward reliable high-confidence reasoning. Concretely, TASCO optimizes a task-level prefix shared across unlabeled test queries to stabilize confidence during reasoning. Once optimized, this prefix is prepended to each query embedding, thereby steering the frozen model throughout trace generation. Since local stability of reasoning confidence can manifest as either variation across nearby perturbed prefixes or sensitivity to the most adversely perturbed prefix, TASCO operationalizes it using two complementary strategies: Random Perturbation measures distributional stability through confidence variation across trajectories independently decoded under Gaussian-perturbed prefixes, while Sharpness-Aware Perturbation targets worst-case local sensitivity by applying a single gradient-guided perturbation along the most sensitive direction in the prefix space for the current trajectory. By optimizing the prefix with these two schemes, TASCO improves accuracy by up to 17.2% while generating up to 28.1% fewer tokens across diverse LLMs and reasoning benchmarks. Behavioral analyses show TASCO promotes stable confidence under local perturbations without prematurely narrowing reasoning directions. The learned prefix improves reasoning on unseen datasets without further optimization.

Our contributions are summarized as follows:

- We introduce a stability-aware perspective on confidence-driven test-time optimization, identifying local stability as a complementary signal for assessing the reliability of high-confidence reasoning.
- We propose TASCO, a label-free framework that uses two alternative perturbation strategies to guide frozen LLMs toward high-confidence states that remain stable under local perturbations. We further characterize how both strategies control local prefix sensitivity.
- Extensive experiments across diverse LLMs and reason-

ing benchmarks demonstrate that TASCO improves reasoning accuracy and efficiency while promoting stable confidence under local perturbations.

Related Work

Reasoning in Large Language Models

Chain-of-thought (CoT) prompting (Wei et al. 2022) improves LLM reasoning by eliciting explicit intermediate steps. Building on this paradigm, subsequent methods enhance reasoning through trajectory aggregation (Wang et al. 2022), structured search (Yao et al. 2023), and step-level verification (Zhang et al. 2025b). Reasoning models further extend explicit CoT by performing longer inference-time deliberation (Chen et al. 2026b). In parallel, continuous reasoning methods move part of the intermediate computation from token space into latent space. COCONUT (Hao et al. 2024) and CODI (Shen et al. 2025), for example, replace explicit reasoning tokens with latent states, while SoftCoT (Xu et al. 2025b) and SemCoT (He et al. 2026) use auxiliary modules to construct continuous rationales that are subsequently aligned with and injected into the target model (Wei et al. 2025). Together, these methods establish latent states as an interface for steering reasoning without backbone updates.

Test-Time Adaptation for Reasoning

Test-time adaptation improves LLM reasoning using inference-time signals. TTRL (Zuo et al. 2026) derives majority-vote pseudo-rewards from rollouts. Entropy-based approaches either use negative token entropy for policy optimization (Prabhudesai et al. 2025) or minimize output entropy to adapt model parameters on unlabeled samples (Gao et al. 2025). Such updates add computation and risk catastrophic forgetting (Hu et al. 2025a). Recent methods instead optimize continuous variables while freezing the model. Some rely on external verifiers or reward models to guide intermediate reasoning states (Han et al. 2026; Chen et al. 2025), whereas others use model-derived entropy to optimize continuous vectors (Kang, Shi, and Chen 2026; Ye, Liang, and Shan 2025). MTI selectively guides high-entropy tokens during decoding (Yang et al. 2026). However, confidence-driven methods rarely assess whether the resulting states are reliable, despite evidence that LLM confidence can be misaligned with correctness (Agarwal et al. 2026). Perturbation-based studies instead use sensitivity diagnostically: CCPS predicts answer correctness from changes in hidden representations (Khanmohammadi et al. 2025), while Wen et al. identify uncertain reasoning steps through embedding perturbations (Wen et al. 2026). Unlike these approaches, TASCO optimizes local stability to promote reliable reasoning.

Method

In this section, we present TASCO, a stability-aware framework that optimizes a continuous prefix at test time while keeping the LLM frozen. We first formulate the optimization problem and define the confidence objective. We then introduce two alternative strategies for characterizing local stability from distributional and worst-case perspectives. We

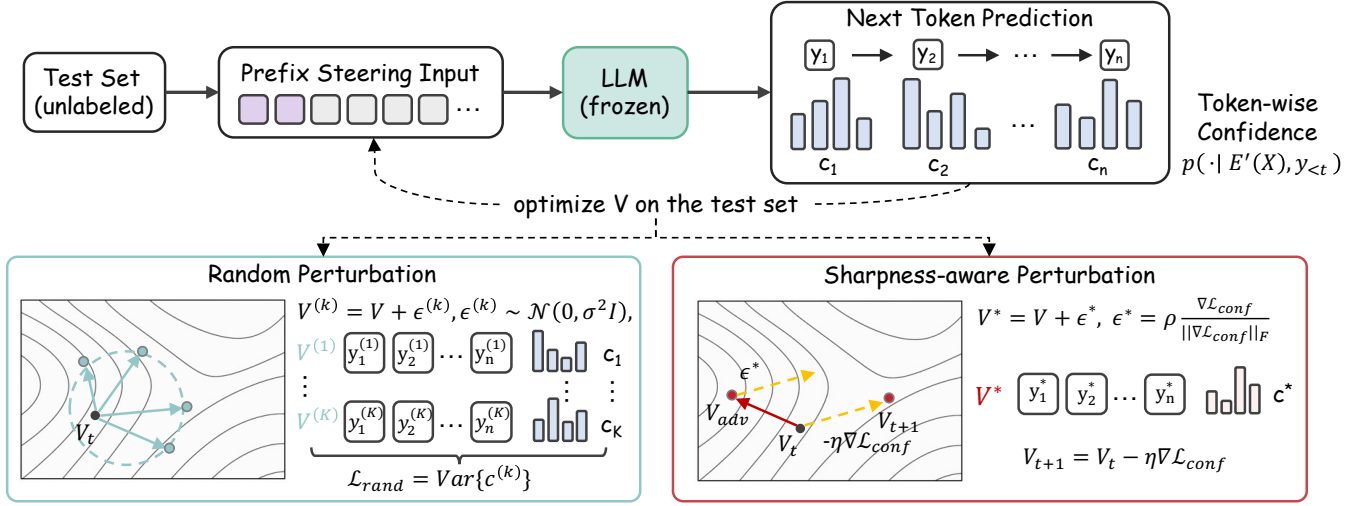


Figure 3: Overview of TASCO. TASCO optimizes a shared prefix for confidence and stability. Random Perturbation penalizes local confidence variance, while Sharpness-Aware Perturbation targets worst-case sensitivity.

further analyze how these strategies guide confidence optimization. Figure 3 provides an overview of TASCO.

Preliminaries

Problem Formulation. Let M_θ denote a pretrained LLM with frozen parameters θ , and let $\mathcal{D}_{\text{test}} = \{X_1, \dots, X_N\}$ be an unlabeled test set. Our goal is to improve the reasoning performance of M_θ on $\mathcal{D}_{\text{test}}$ without updating its parameters. To this end, we optimize a shared continuous prefix $V \in \mathbb{R}^{L \times d}$. Here, L and d denote the prefix length and embedding dimension. Given a query X with token embeddings $E(X)$, we prepend V to form the prefix-conditioned input $E'(X) = [V; E(X)]$, using $[\cdot; \cdot]$ to denote concatenation along the sequence dimension. The frozen model then autoregressively generates a reasoning trajectory $Y = (y_1, \dots, y_T)$. At decoding step t , its next-token distribution is $p_\theta(\cdot | E'(X), y_{<t})$. TASCO optimizes V using only label-free signals derived from M_θ , without external supervision or auxiliary models.

Confidence Objective. We use predictive entropy as a label-free confidence signal. For a generated trajectory $Y = (y_1, \dots, y_T)$, the confidence loss is defined as the average token-level entropy:

$$\mathcal{L}_{\text{conf}}(V; X, Y) = \frac{1}{T} \sum_{t=1}^T H(p_\theta(\cdot | E'(X), y_{<t})), \quad (1)$$

where $H(p) = -\sum_{w \in \mathcal{V}} p(w) \log p(w)$ denotes the entropy over the vocabulary $\mathcal{V}$. In practice, we compute this loss using one generated trajectory per input and treat the generated tokens as fixed.

Confidence Optimization with Local Stability

Although entropy minimization offers a label-free objective for prefix optimization, it may steer the prefix toward a locally fragile high-confidence state. TASCO therefore introduces

two local-stability strategies: (1) Random Perturbation for distributional stability and (2) Sharpness-Aware Perturbation for worst-case directional stability.

Random Perturbation. We promote distributional stability by examining reasoning trajectories decoded under nearby prefixes. At each optimization step, we draw K Gaussian perturbations around the current prefix:

$$V^{(k)} = V + \epsilon^{(k)}, \quad \epsilon^{(k)} \sim \mathcal{N}(0, \sigma^2 I), \quad (2)$$

where σ is the standard deviation of the Gaussian noise applied to each prefix element. For each input X , the model greedily decodes a trajectory $Y^{(k)}$ under every perturbed prefix $V^{(k)}$. We summarize each rollout by the average log-likelihood of its generated tokens:

$$c^{(k)} = \frac{1}{T_k} \sum_{t=1}^{T_k} \log p_\theta(y_t^{(k)} | E'^{(k)}(X), y_{<t}^{(k)}). \quad (3)$$

Variation in $\{c^{(k)}\}_{k=1}^K$ reflects local behavioral instability and may capture trajectory switching because each perturbed prefix is re-decoded. We quantify the behavioral sharpness of V on X by the sample variance:

$$\mathcal{S}(V; X) = \frac{1}{K-1} \sum_{k=1}^K (c^{(k)} - \bar{c})^2, \quad \bar{c} = \frac{1}{K} \sum_{k=1}^K c^{(k)}. \quad (4)$$

For a mini-batch $\mathcal{B}$, the random-perturbation regularizer is obtained by averaging this quantity across inputs, $\mathcal{L}_{\text{rand}}(V) = |\mathcal{B}|^{-1} \sum_{X \in \mathcal{B}} \mathcal{S}(V; X)$. Combining it with the batch-averaged confidence loss gives:

$$\mathcal{L}_{\text{RP}} = \mathcal{L}_{\text{conf}}(V) + \lambda_{\text{rand}} \mathcal{L}_{\text{rand}}(V), \quad (5)$$

where λ_{rand} balances predictive confidence and local stability. Thus, $\mathcal{L}_{\text{conf}}$ encourages low predictive entropy, whereas $\mathcal{L}_{\text{rand}}$ reduces behavioral variation across nearby prefixes.

Sharpness-Aware Perturbation. Unlike Random Perturbation, Sharpness-Aware Perturbation avoids generating multiple trajectories by constructing a single gradient-guided perturbation. At each update, we fix the trajectory Y decoded under V while minimizing the worst-case confidence loss:

$$\min_V \max_{\|\epsilon\|_F \leq \rho} \mathcal{L}_{\text{conf}}(V + \epsilon; X, Y), \quad (6)$$

where ρ defines the perturbation radius under the Frobenius norm. Since the inner maximization in Eq. 6 is generally intractable, we use a first-order approximation of the confidence loss around V , based on

$$g = \nabla_V \mathcal{L}_{\text{conf}}(V; X, Y). \quad (7)$$

The linearized inner problem yields the following worst-case perturbation and perturbed prefix:

$$\hat{V} = V + \epsilon^*, \quad \epsilon^* = \rho \frac{g}{\|g\|_F}, \quad (8)$$

where ϵ^* is the perturbation along the steepest ascent direction under $\|\epsilon\|_F \leq \rho$. We then evaluate the confidence loss at $\hat{V}$ using the same trajectory Y :

$$\mathcal{L}_{\text{SAP}} = \mathcal{L}_{\text{conf}}(\hat{V}; X, Y). \quad (9)$$

When updating V with $\mathcal{L}_{\text{SAP}}$, we treat ϵ^* as constant to avoid second-order derivatives through g .

The variants trade off neighborhood coverage and computation: Random Perturbation captures variation across sampled perturbations, whereas SAP targets worst-case sensitivity with one gradient-guided perturbation. Complete algorithms appear in Appendix D.

Theoretical Analysis

We analyze how the two objectives control local confidence sensitivity to prefix perturbations. Random Perturbation captures confidence variation across perturbed rollouts, whereas SAP targets worst-case sensitivity along a fixed trajectory.

Random Perturbation For $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$, let Z denote the greedy trajectory decoded under $V + \epsilon$, and let $C = c(V + \epsilon)$ denote its trajectory-level confidence. The sample variance in Eq. 23 is an unbiased estimator of the following population behavioral sharpness:

$$\mathcal{S}_\sigma(V; X) = \text{Var}_{\epsilon \sim \mathcal{N}(0, \sigma^2 I)} [c(V + \epsilon)]. \quad (10)$$

Because each perturbed prefix is independently re-decoded, the law of total variance gives:

$$\mathcal{S}_\sigma(V; X) = \mathbb{E}_Z [\text{Var}(C | Z)] + \text{Var}_Z [\mathbb{E}[C | Z]]. \quad (11)$$

The first term captures variation among perturbations yielding the same trajectory, whereas the second captures variation in mean confidence across decoded trajectories, without differentiating through decoding.

For the within-trajectory term, consider a local neighborhood in which greedy decoding remains Y , and let $f_Y(V)$ denote the teacher-forced confidence of Y in Eq. 22. If f_Y is sufficiently smooth, then, as $\sigma \rightarrow 0$:

$$\text{Var}_\epsilon [f_Y(V + \epsilon)] = \sigma^2 \|\nabla_V f_Y(V)\|_F^2 + \mathcal{O}(\sigma^4). \quad (12)$$

Thus, away from stationary points, Random Perturbation penalizes first-order confidence sensitivity to isotropic perturbations. At a stationary point of f_Y , the corresponding expansion is:

$$\text{Var}_\epsilon [f_Y(V + \epsilon)] = \frac{\sigma^4}{2} \|\nabla_V^2 f_Y(V)\|_F^2 + o(\sigma^4). \quad (13)$$

The latter reflects aggregate local curvature, while Eq. 11 additionally captures between-trajectory variation.

Sharpness-Aware Perturbation For a trajectory Y held fixed within an update, define $\ell(V) = \mathcal{L}_{\text{conf}}(V; X, Y)$. Its worst-case local increase over a Frobenius ρ -ball is:

$$\Delta_\rho(V) = \max_{\|\epsilon\|_F \leq \rho} [\ell(V + \epsilon) - \ell(V)]. \quad (14)$$

Assume that ℓ has a β -Lipschitz-continuous gradient on this ball, and let $g = \nabla_V \ell(V)$. Then:

$$|\Delta_\rho(V) - \rho \|g\|_F| \leq \frac{\beta \rho^2}{2}. \quad (15)$$

Thus, the gradient norm provides a first-order approximation to the worst-case local increase. For $g \neq 0$, the perturbation ϵ^* in Eq. 8 satisfies:

$$|\ell(V + \epsilon^*) - \ell(V) - \rho \|g\|_F| \leq \frac{\beta \rho^2}{2}. \quad (16)$$

Together, Eqs. 15 and 16 show that the induced increase $\ell(V + \epsilon^*) - \ell(V)$ approximates $\Delta_\rho(V)$ with $\mathcal{O}(\rho^2)$ error.

Separately, if V is a stationary point and ℓ is twice continuously differentiable in a neighborhood of V , the ideal inner problem admits the following expansion as $\rho \rightarrow 0$:

$$\Delta_\rho(V) = \frac{\rho^2}{2} [\lambda_{\max}(\nabla_V^2 \ell(V))]_+ + o(\rho^2), \quad (17)$$

where $[a]_+ = \max(a, 0)$. Thus, the leading term of the ideal worst-case increase is governed by the largest positive local curvature.

Together, the two objectives control local confidence sensitivity under sampled and worst-case perturbations. These results characterize robustness rather than correctness; the empirical relationship between stability and answer accuracy is examined in Appendix B.

Experiments

Experimental Setup

Models. We evaluate TASCO on five open-source LLMs covering both general and reasoning models. General LLMs include Qwen2.5-Math (Yang et al. 2024) in two sizes (1.5B and 7B) and LLaMA3.1-8B-Instruct (Grattafiori et al. 2024). Reasoning LLMs include DeepSeek-R1-Distill-Qwen in two sizes (1.5B and 7B) (Guo et al. 2025).

Datasets. We evaluate on six reasoning benchmarks. For mathematics, we use MATH-500 (Hendrycks et al. 2021), AMC23, AIME24 (Li et al. 2024), and AIME25 (Balunovic et al. 2026), spanning high-school to olympiad-level problems. For science, we use Minerva Math (Lewkowycz et al. 2022), with undergraduate-level STEM problems, and GPQA Diamond (Rein et al. 2023), with graduate-level questions written by domain experts.

Model	Method	MATH500	AMC23	Minerva	AIME24	GPQA	Avg.	Δ
Qwen2.5-Math-1.5B	Zero-Shot CoT	39.4	34.7	8.8	6.7	20.2	22.0	–
	SLOT	43.4	47.2	16.9	10.0	<u>26.8</u>	28.9	+6.9
	LatentSeek	45.6	43.4	14.3	7.5	22.2	26.6	+4.6
	LTPO	58.6	45.6	12.9	9.2	21.7	29.6	+7.6
	TTSV	68.6	46.8	20.6	8.8	24.8	33.9	+11.9
	TASCO-RP	70.8	50.6	22.4	<u>10.4</u>	27.8	36.4	+14.4
	TASCO-SAP	71.8	52.2	<u>21.3</u>	12.1	<u>26.8</u>	36.8	+14.8
Qwen2.5-Math-7B	Zero-Shot CoT	52.2	44.4	12.9	14.6	31.8	31.2	–
	SLOT	58.8	56.2	29.4	18.8	32.8	39.2	+8.0
	LatentSeek	57.6	48.1	20.5	6.2	32.8	33.0	+1.8
	LTPO	65.6	42.8	17.6	18.8	33.5	35.7	+4.5
	TTSV	71.0	59.1	33.5	17.9	36.4	43.6	+12.4
	TASCO-RP	75.8	68.8	<u>37.1</u>	<u>21.2</u>	<u>37.4</u>	<u>48.1</u>	+16.9
	TASCO-SAP	<u>73.4</u>	<u>66.2</u>	39.1	24.2	38.9	48.4	+17.2
Llama3.1-8B-Instruct	Zero-Shot CoT	47.4	24.7	22.1	7.5	28.3	26.0	–
	SLOT	48.8	23.8	22.8	9.2	33.8	27.7	+1.7
	LatentSeek	49.4	26.8	23.2	3.3	32.3	27.0	+1.0
	LTPO	48.4	27.5	20.5	<u>11.3</u>	30.5	27.6	+1.6
	TTSV	47.8	22.8	22.1	6.2	27.8	25.3	-0.7
	TASCO-RP	46.8	29.1	<u>25.0</u>	10.4	<u>33.3</u>	<u>28.9</u>	+2.9
	TASCO-SAP	50.2	<u>28.4</u>	26.5	11.7	31.8	29.7	+3.7

Table 1: Main performance comparison across three models and five reasoning benchmarks (accuracy, %). Δ denotes the absolute percentage-point change relative to Zero-Shot CoT. Best results are in **bold**; second-best results are underlined.

Baselines. For general LLMs, we include CoT (Wei et al. 2022) as the standard prompting baseline. We compare with SLOT (Hu et al. 2025b) and LatentSeek (Li et al. 2025), which perform test-time adaptation through final-layer or latent representations. We also compare with LTPO (Ye, Liang, and Shan 2025) and TTSV (Kang, Shi, and Chen 2026), which optimize continuous latent variables using model-internal confidence signals while keeping the backbone frozen. For reasoning LLMs, we compare with s1 (Muenighoff et al. 2025), CoD (Xu et al. 2025a), and $\alpha 1$ (Zhang et al. 2025a), which control reasoning behavior at test time.

Implementation Details. For general LLMs, we use a shared prefix of length $L = 20$ and a maximum generation length of 3,072 tokens. For reasoning-enhanced LLMs, we use $L = 10$ and allow up to 8,192 tokens for longer reasoning traces. We optimize the prefix using AdamW with a batch size of 16. Random Perturbation uses $K = 8$ perturbed rollouts with $\sigma = 0.01$, while Sharpness-Aware Perturbation uses a perturbation radius of $\rho = 0.8$. We further analyze sensitivity to σ and ρ . All experiments run on a single NVIDIA A800 GPU. More details are provided in Appendix A.

Evaluation. We report accuracy on all benchmarks. For mathematical reasoning benchmarks, we extract the final answer and determine correctness using dataset-specific normalization and equivalence rules. For AMC23 and AIME24, we independently sample eight responses for each problem and report the mean accuracy across the 8 generations.

Main Results

Tables 1 and 2 compare TASCO with baselines on general and reasoning-enhanced LLMs, respectively, showing accuracy gains across both model types. Furthermore, steering

the model toward high-confidence reasoning states yields shorter outputs, improving token efficiency.

Performance Improvement. Tables 1 and 2 show TASCO improves both general and reasoning-enhanced LLMs. On Qwen2.5-Math-1.5B and Qwen2.5-Math-7B, TASCO raises average accuracy from 22.0% to 36.8% and from 31.2% to 48.4%, gains of 14.8 and 17.2 percentage points over Zero-Shot CoT. TASCO further outperforms TTSV by 2.9 and 4.8 points, respectively. TASCO also improves LLaMA3.1-8B-Instruct from 26.0% to 29.7%, demonstrating applicability beyond Qwen. On reasoning-enhanced models, TASCO improves the four-benchmark average from 45.8% to 52.6% on DeepSeek-R1-Distill-Qwen-1.5B and from 59.5% to 65.5% on DeepSeek-R1-Distill-Qwen-7B, gains of 6.8 and 6.0 percentage points. It surpasses the strongest reasoning-control baseline by 4.3 and 2.5 points, respectively. Overall, incorporating local stability into confidence optimization improves test-time reasoning across model families and reasoning paradigms.

Token Efficiency. TASCO also improves token efficiency. Figure 4 shows that TASCO-R and TASCO-S reduce average generation length by 28.1% and 24.0%, respectively, across general LLMs compared with CoT. Similar gains on reasoning-enhanced LLMs are reported in Appendix E. These reductions lower inference cost and suggest that TASCO improves accuracy without requiring longer reasoning traces. Instead, the optimized prefix promotes concise, reliable reasoning while improving accuracy.

Analyses of TASCO

Ablation of Prefix Optimization. We exam whether TASCO’s gains arise from using local stability to improve

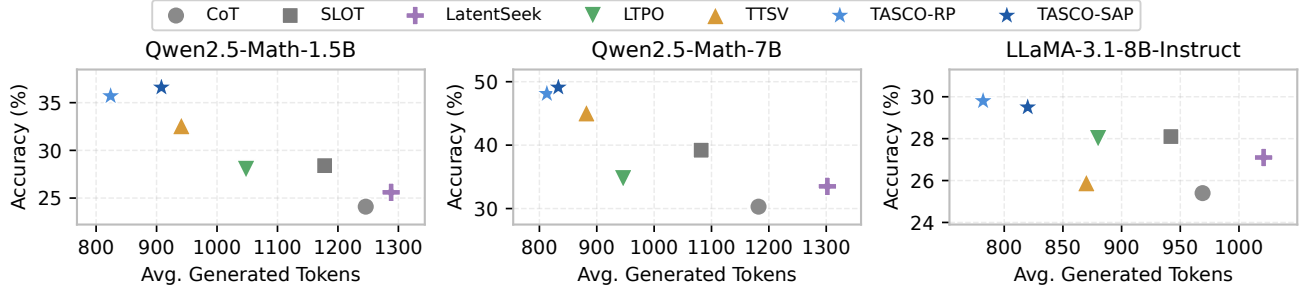


Figure 4: Accuracy-token trade-off across general LLMs, averaged over all benchmarks.

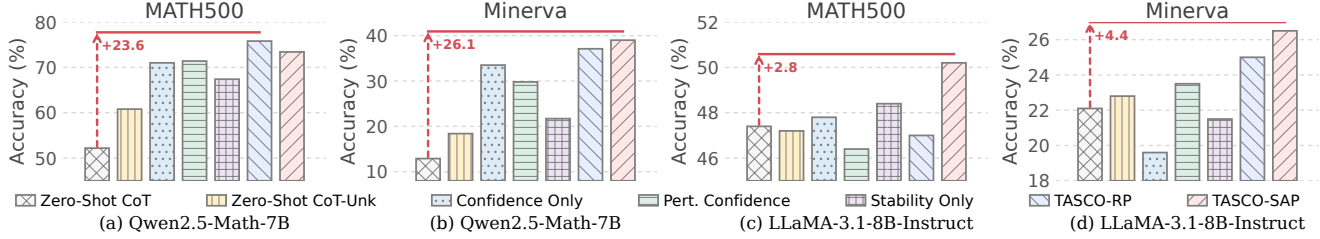


Figure 5: Impact of optimized prefixes on MATH500 and Minerva. The results indicate that the gains of TASCO come from learning stability-aware task-level prefixes rather than simply adding extra prefix positions.

Method	AIME24	AIME25	AMC23	MATH500
<i>DeepSeek-R1-Distill-Qwen-1.5B</i>				
Base	22.8	22.5	59.4	78.6
s1	25.8	25.4	61.9	80.2
CoD	26.7	20.8	66.2	79.4
α_1	30.8	12.9	63.1	81.0
TASCO	30.4	29.2	69.4	81.4
<i>DeepSeek-R1-Distill-Qwen-7B</i>				
Base	44.2	25.0	79.4	89.4
s1	37.5	24.2	78.1	89.0
CoD	45.0	33.3	85.6	88.0
α_1	36.7	32.5	90.6	89.8
TASCO	49.2	34.2	88.1	90.4

Table 2: Evaluation on reasoning-enhanced LLMs using DeepSeek-R1-Distill models as backbones. TASCO results are obtained with the TASCO-SAP variant.

confidence-guided reasoning. CoT-Unk and Confidence Only test whether additional prefix positions or confidence optimization alone suffice. Perturbed Confidence uses the same K trajectories as TASCO-RP but removes the stability objective, isolating additional rollout computation; Stability Only removes the confidence objective, testing whether stability regularization alone explains the gains. As shown in Fig. 5, TASCO-RP outperforms both controlled variants. Together with the improvements of both TASCO variants over Confidence Only, these results show that the gains arise neither from additional computation nor from stability regularization in isolation, but from using local stability to guide optimization toward reliable high-confidence reasoning.

Behavioral Effects of Stability-Aware Optimization. We examine how TASCO affects local robustness and confidence formation on Qwen2.5-Math-7B. To assess local robustness, we add Gaussian noise with $\sigma = 0.01$ to each learned prefix and generate $K = 8$ perturbed rollouts per input. As shown in Table 3, confidence-only optimization produces higher trajectory-confidence variance and lower answer consistency across datasets, whereas TASCO reduces this variation and improves consistency. We further analyze confidence formation on MATH500 by tracking the mean next-token entropy and top-1 probability over the first 5% of decoding steps. Figure 7 shows that confidence-only optimization rapidly concentrates probability mass early in generation, whereas TASCO retains higher entropy and lower top-1 probability before the gap narrows later. Together, these results show that TASCO produces reasoning behavior that is more stable under local perturbations and less prone to premature confidence concentration.

Analysis of Cross-Distribution Generalization. To assess the transferability of the guidance learned by TASCO, we optimize a prefix on one source benchmark and directly apply it to target benchmarks without re-optimization. As shown in Fig. 6, the transferred prefixes consistently outperform Zero-Shot CoT across all evaluated source-target pairs, indicating that the learned guidance is not confined to its optimization distribution. This transferability stems from TASCO’s stability-aware objective: by promoting confidence that remains stable under local perturbations, TASCO guides the model to learn reliable reasoning patterns shared across mathematical tasks rather than merely increasing confidence on the source distribution.

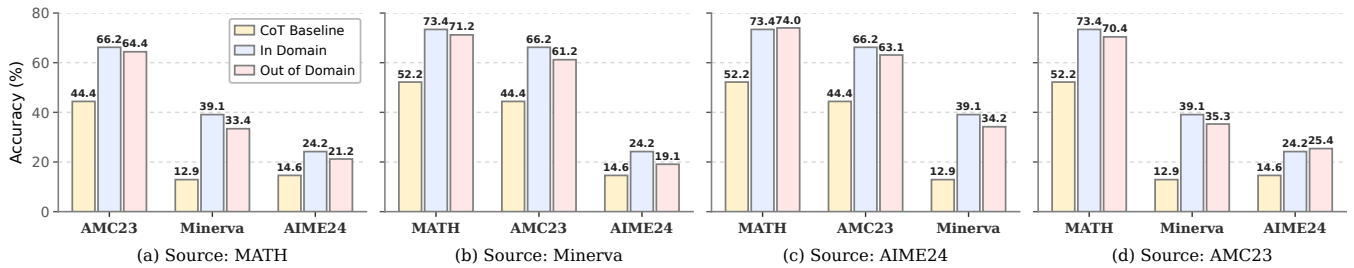


Figure 6: Cross-distribution generalization of TASCO on Qwen2.5-Math-7B, measured by accuracy gains over Zero-Shot CoT.

Method	MATH500	AMC23	Minerva	GPQA
<i>Confidence Variance ($\times 10^{-4}$) $\downarrow$</i>				
Confidence Only	4.83	3.05	6.59	15.76
TASCO-RP	2.33	2.34	5.48	9.31
TASCO-SAP	2.89	2.32	4.51	12.26
<i>Answer Consistency $\uparrow$</i>				
Confidence Only	62.9	42.5	58.9	48.1
TASCO-RP	71.7	47.0	62.6	59.4
TASCO-SAP	73.8	50.5	66.6	52.8

Table 3: Local stability under prefix perturbations on Qwen2.5-Math-7B. Lower variance and higher consistency indicate more stable reasoning behavior.

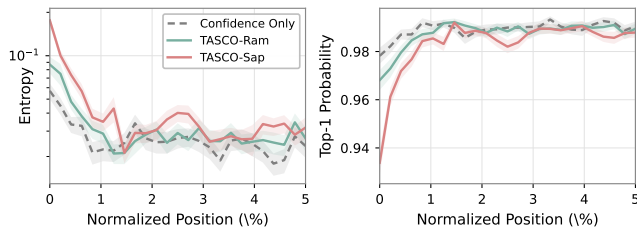


Figure 7: Mean next-token entropy (left) and top-1 probability (right) over the first 5% of decoding steps for Qwen2.5-Math-7B on MATH500.

Performance Across Query Difficulty. We first estimate query difficulty from eight responses sampled by the unadapted model: queries answered correctly 0–2, 3–5, and 6–8 times are categorized as hard, medium, and easy, respectively. The resulting groups contain 195, 130, and 175 queries for Qwen2.5-Math-7B and 216, 92, and 192 queries for LLaMA3.1-8B-Instruct. We then compare TASCO with confidence-only optimization within the same groups. As shown in Table 4, TASCO yields clear gains on hard and medium queries across both models while maintaining comparable performance on easy queries. These results suggest that TASCO does more than reinforce solutions the model already finds easy; it learns reasoning guidance that supports more reliable reasoning on challenging problems.

Sensitivity to Perturbation Scale. We evaluate sensitivity to the perturbation scale on Qwen2.5-Math-1.5B with MATH500. For Random Perturbation, we vary the Gaussian

Difficulty	Qwen2.5-Math-7B		LLaMA3.1-8B	
	Confidence	TASCO	Confidence	TASCO
Hard	46.7	54.4	9.7	14.4
Medium	75.4	80.8	48.9	54.3
Easy	94.9	96.0	90.1	88.5
Overall	71.0	75.8	47.8	50.2

Table 4: Accuracy (%) of confidence-only optimization and TASCO-RP across MATH500 difficulty groups.



Figure 8: Sensitivity of TASCO to the perturbation scale on Qwen2.5-Math-1.5B with MATH500.

scale σ ; for Sharpness-Aware Perturbation, we vary the radius ρ , while fixing all other settings. Figure 8 shows that both variants outperform confidence-only optimization across a broad range of nonzero scales. Accuracy peaks near the selected defaults and decreases only when the perturbation becomes excessively large, indicating that TASCO does not rely on narrow hyperparameter tuning. Additional sensitivity analyses are provided in Appendix E.

Conclusion

We observed that high-confidence reasoning is more likely to yield correct answers when it remains stable under local perturbations. Based on this observation, we proposed TASCO, a label-free framework that optimizes a shared task-level prefix for confidence and stability while keeping the LLM frozen. Random Perturbation captures distributional variation across nearby behaviors, whereas Sharpness-Aware Perturbation targets worst-case local sensitivity. Experiments across diverse LLMs and reasoning benchmarks show that stability-aware confidence optimization improves reasoning accuracy and generation efficiency without parameter updates or external supervision.

References

- Agarwal, S.; Zhang, Z.; Yuan, L.; Han, J.; and Peng, H. 2026. The unreasonable effectiveness of entropy minimization in llm reasoning. *Advances in Neural Information Processing Systems*, 38: 107150–107180.
- Balunovic, M.; Dekoninck, J.; Petrov, I.; Jovanović, N.; and Vechev, M. 2026. Matharena: Evaluating llms on uncontaminated math competitions. *Advances in Neural Information Processing Systems*, 38.
- Chen, G.; Niu, S.; Chen, D.; Yang, J.; Zhang, Z.; Tan, M.; Wu, P.; and Shen, Z. 2026a. ZeroSiam: An Efficient Asymmetry for Test-Time Entropy Optimization without Collapse. In *The Fourteenth International Conference on Learning Representations*.
- Chen, Q.; Qin, L.; Liu, J.; Peng, D.; Guan, J.; Wang, P.; Hu, M.; Zhou, Y.; Gao, T.; and Che, W. 2026b. Towards reasoning era: A survey of long chain-of-thought for reasoning large language models. *Science China Information Sciences*, 69(6): 161101.
- Chen, R.; Zhang, Z.; Hong, J.; Kundu, S.; and Wang, Z. 2025. Seal: Steerable reasoning calibration of large language models for free.
- Gao, Z.; Chen, L.; Luo, H.; Zhou, J.; and Dai, B. 2025. One-shot entropy minimization.
- Grattafiori, A.; Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Vaughan, A.; et al. 2024. The llama 3 herd of models.
- Guo, D.; Yang, D.; Zhang, H.; Song, J.; Wang, P.; Zhu, Q.; Xu, R.; Zhang, R.; Ma, S.; Bi, X.; et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning.
- Han, P.; Xu, X.; Xuan, K.; Song, P.; Ouyang, S.; Tian, R.; Jiang, Y.; Qian, C.; Jiang, P.; Sun, J.; et al. 2026. Steer2Adapt: Dynamically Composing Steering Vectors Elicits Efficient Adaptation of LLMs.
- Hao, S.; Sukhbaatar, S.; Su, D.; Li, X.; Hu, Z.; Weston, J.; and Tian, Y. 2024. Training large language models to reason in a continuous latent space.
- He, Y.; Zheng, W.; Zhu, Y.; Zheng, Z.; Su, L.; Vasudevan, S.; Guo, Q.; Hong, L.; and Li, J. 2026. SemCoT: Accelerating Chain-of-Thought Reasoning through Semantically-Aligned Implicit Tokens. *Advances in Neural Information Processing Systems*, 38: 43455–43485.
- Hendrycks, D.; Burns, C.; Kadavath, S.; Arora, A.; Basart, S.; Tang, E.; Song, D.; and Steinhardt, J. 2021. Measuring mathematical problem solving with the math dataset.
- Hu, J.; Zhang, Z.; Chen, G.; Wen, X.; Shuai, C.; Luo, W.; Xiao, B.; Li, Y.; and Tan, M. 2025a. Test-time learning for large language models.
- Hu, Y.; Zhang, X.; Fang, X.; Chen, Z.; Wang, X.; Zhang, H.; and Qi, G. 2025b. Slot: Sample-specific language model optimization at test-time.
- Huang, C.; Huang, L.; Leng, J.; Liu, J.; and Huang, J. 2025. Efficient test-time scaling via self-calibration.
- Jaech, A.; Kalai, A.; Lerer, A.; Richardson, A.; El-Kishky, A.; Low, A.; Helyar, A.; Madry, A.; Beutel, A.; Carney, A.; et al. 2024. Openai o1 system card.
- Jurayj, W.; Cheng, J.; and Van Durme, B. 2025. Is that your final answer? test-time scaling improves selective question answering. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 636–644.
- Kang, X.; Shi, D.; and Chen, L. 2026. Model Whisper: Steering Vectors Unlock Large Language Models’ Potential in Test-Time. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 31392–31400.
- Keskar, N. S.; Mudigere, D.; Nocedal, J.; Smelyanskiy, M.; and Tang, P. T. P. 2016. On large-batch training for deep learning: Generalization gap and sharp minima.
- Khalifa, M.; Agarwal, R.; Logeswaran, L.; Kim, J.; Peng, H.; Lee, M.; Lee, H.; and Wang, L. 2025. Process reward models that think.
- Khanmohammadi, R.; Miahi, E.; Mardikoraem, M.; Kaur, S.; Brugere, I.; Smiley, C.; Thind, K. S.; and Ghassemi, M. M. 2025. Calibrating LLM confidence by probing perturbed representation stability. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 10459–10525.
- Lambert, N.; Morrison, J.; Pyatkin, V.; Huang, S.; Ivison, H.; Brahman, F.; Miranda, L. J. V.; Liu, A.; Dziri, N.; Lyu, S.; et al. 2024. Tulu 3: Pushing frontiers in open language model post-training.
- Lewkowycz, A.; Andreassen, A.; Dohan, D.; Dyer, E.; Michalewski, H.; Ramasesh, V.; Slone, A.; Anil, C.; Schlag, I.; Gutman-Solo, T.; et al. 2022. Solving quantitative reasoning problems with language models. *Advances in neural information processing systems*, 35: 3843–3857.
- Li, H.; Li, C.; Wu, T.; Zhu, X.; Wang, Y.; Yu, Z.; Jiang, E. H.; Zhu, S.-C.; Jia, Z.; Wu, Y. N.; et al. 2025. Seek in the dark: Reasoning via test-time instance-level policy gradient in latent space. *arXiv preprint arXiv:2505.13308*.
- Li, J.; Beeching, E.; Tunstall, L.; Lipkin, B.; Soletskyi, R.; Huang, S.; Rasul, K.; Yu, L.; Jiang, A. Q.; Shen, Z.; et al. 2024. Numinamath: The largest public dataset in ai4maths with 860k pairs of competition math problems and solutions. *Hugging Face repository*, 13(9): 9.
- Muennighoff, N.; Yang, Z.; Shi, W.; Li, X. L.; Fei-Fei, L.; Hajishirzi, H.; Zettlemoyer, L.; Liang, P.; Candès, E.; and Hashimoto, T. B. 2025. s1: Simple test-time scaling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 20286–20332.
- Nguyen, T.; and Le, T. 2026. ATLAS: Adaptive Test-Time Latent Steering with External Verifiers for Enhancing LLMs Reasoning.
- Prabhudesai, M.; Chen, L.; Ippoliti, A.; Fragkiadaki, K.; Liu, H.; and Pathak, D. 2025. Maximizing confidence alone improves reasoning.
- Rein, D.; Hou, B. L.; Stickland, A. C.; Petty, J.; Pang, R. Y.; Dirani, J.; Michael, J.; and Bowman, S. R. 2023. Gpqa: A graduate-level google-proof q&a benchmark.

- Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Bi, X.; Zhang, H.; Zhang, M.; Li, Y.; Wu, Y.; et al. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models.
- Shen, Z.; Yan, H.; Zhang, L.; Hu, Z.; Du, Y.; and He, Y. 2025. Codi: Compressing chain-of-thought into continuous space via self-distillation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 677–693.
- Sinii, V.; Gorbatoyski, A.; Cherepanov, A.; Shaposhnikov, B.; Balagansky, N.; and Gavrilov, D. 2025. Steering llm reasoning through bias-only adaptation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 9213–9222.
- Wang, X.; Wei, J.; Schuurmans, D.; Le, Q.; Chi, E.; Narang, S.; Chowdhery, A.; and Zhou, D. 2022. Self-consistency improves chain of thought reasoning in language models.
- Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Xia, F.; Chi, E.; Le, Q. V.; Zhou, D.; et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35: 24824–24837.
- Wei, X.; Liu, X.; Zang, Y.; Dong, X.; Cao, Y.; Wang, J.; Qiu, X.; and Lin, D. 2025. SIM-CoT: Supervised Implicit Chain-of-Thought.
- Wen, Q.; Wang, J.; Nan, Y.; He, P.; Tandon, R.; and Xu, H. 2026. Embedding Perturbation may Better Reflect the Uncertainty in LLM Reasoning.
- Xu, S.; Xie, W.; Zhao, L.; and He, P. 2025a. Chain of draft: Thinking faster by writing less.
- Xu, Y.; Guo, X.; Zeng, Z.; and Miao, C. 2025b. Softcot: Soft chain-of-thought for efficient reasoning with llms. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 23336–23351.
- Yang, A.; Zhang, B.; Hui, B.; Gao, B.; Yu, B.; Li, C.; Liu, D.; Tu, J.; Zhou, J.; Lin, J.; et al. 2024. Qwen2. 5-math technical report: Toward mathematical expert model via self-improvement.
- Yang, Z.; Zhang, M.; Chen, F.; Ding, G.; Hou, L.; Tao, X.; and Chen, Y.-C. 2026. Less is more: Improving llm reasoning with minimal test-time intervention. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 20124–20137.
- Yao, S.; Yu, D.; Zhao, J.; Shafran, I.; Griffiths, T.; Cao, Y.; and Narasimhan, K. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36: 11809–11822.
- Ye, W.; Liang, Y.; and Shan, L. 2025. Thinking on the Fly: Test-Time Reasoning Enhancement via Latent Thought Policy Optimization. *arXiv preprint arXiv:2510.04182*.
- Zhang, J.; Dong, R.; Wang, H.; Ning, X.; Geng, H.; Li, P.; He, X.; Bai, Y.; Malik, J.; Gupta, S.; et al. 2025a. Alphaone: Reasoning models thinking slow and fast at test time. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 11340–11365.
- Zhang, Y.; Wang, X.; Wu, L.; and Wang, J. 2025b. Enhancing chain of thought prompting in large language models via reasoning patterns. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, 25985–25993.
- Zhao, C.; Yang, E.; Liu, Y.; Zhao, J.; and Guo, G. 2026. ECHO: Entropy-Confidence Hybrid Optimization for Test-Time Reinforcement Learning.
- Zuo, Y.; Zhang, K.; Sheng, L.; Qu, S.; Cui, G.; Zhu, X.; Li, H.; Long, X.; Hua, E.; Qi, B.; et al. 2026. Ttrl: Test-time reinforcement learning. *Advances in Neural Information Processing Systems*, 38: 131459–131483.

Appendix

A. Experimental Details

This section provides the implementation details of TASCO and the compared baselines. We first describe the prefix optimization and perturbation procedures, followed by the baseline implementations and evaluation protocol.

Optimization and Generation Settings. We optimize only the continuous prefix V using AdamW while keeping all parameters of M_θ frozen. During optimization, trajectories are generated using the model-specific temperature, top- p , and repetition penalty reported in Table 5. The confidence objective is computed from at most the first T_{opt} generated tokens, reported as “Conf. tokens.” Final evaluation uses greedy decoding subject to the corresponding maximum generation length.

Prefix Initialization. TASCO can be optimized directly from a freshly initialized prefix. We use model-family-specific initialization. Prefixes for Qwen-based models, including DeepSeek-R1-Distill-Qwen, are sampled from $\mathcal{N}(0, I)$. For LLaMA models, we first encode the unlabeled optimization set and compute the per-dimension empirical mean and variance of the resulting token embeddings. Each prefix vector is then sampled from the Gaussian distribution parameterized by these statistics. Since perturbation-based stability optimization requires additional rollouts, applying it before the prefix has been guided toward higher-confidence reasoning may be inefficient. In addition to direct optimization from scratch, we also support an initial confidence-only stage of 10 epochs, followed by five epochs of TASCO optimization. All stages use the same learning rate reported in Table 5.

Perturbation Settings. For Random Perturbation, we draw $K = 8$ Gaussian perturbations with $\sigma = 0.01$ at each optimization step. We set the stability weight to $\lambda_{\text{rand}} = 20$ for Qwen2.5-Math and $\lambda_{\text{rand}} = 5$ for LLaMA3.1-Instruct and DeepSeek-R1-Distill-Qwen. For Sharpness-Aware Perturbation, we use a perturbation radius of $\rho = 0.8$. The two variants are optimized and evaluated separately.

Trajectory Generation and Updates. At each optimization step, the model generates one trajectory for every input in the current mini-batch using the current prefix. The generated tokens are held fixed during backpropagation, and gradients are computed only with respect to V . New trajectories are generated after each prefix update. Random Perturbation independently decodes a trajectory under each perturbed prefix, whereas SAP evaluates its gradient-guided perturbation using the trajectory fixed within the current update.

Baseline Implementation Details

We use the official implementations and follow the configurations reported by the original authors whenever available.

Chain-of-Thought. CoT elicits explicit step-by-step reasoning through natural-language instructions without modifying the model.

SLOT. SLOT adapts each test query by optimizing a sample-specific vector added to the model’s final hidden representations.

LatentSeek. LatentSeek uses policy gradients and self-generated rewards to iteratively optimize instance-specific latent reasoning representations.

LTPO. LTPO optimizes latent thought vectors at test time using an intrinsic confidence reward while keeping the LLM frozen.

TTSV. TTSV learns a task-level continuous prefix by minimizing predictive entropy over the unlabeled test set.

s1. s1 applies budget forcing to extend reasoning by inserting additional `wait` tokens before the model terminates its thinking process.

Chain of Draft. CoD encourages concise reasoning by prompting the model to express each intermediate step using only a few words.

$\alpha 1$. $\alpha 1$ controls the transition from slow to fast reasoning through a dynamically scheduled α -moment.

Prompt Design. We use fixed model-family-specific prompt templates throughout optimization and evaluation. Table 11 provides a complete example of the `qwen25-math-cot` prompt, the embedding-level prefix placement, and the resulting response format. The remaining templates are included in the released implementation.

Evaluation. For MATH500, Minerva Math, and GPQA, the main tables report results from a single run. For AMC23 and AIME24, whose smaller test sets produce greater accuracy fluctuations, we independently run each experiment with eight random seeds and report the mean accuracy.

B. Exploring Local Confidence Stability

We investigate whether local confidence stability is associated with answer correctness and whether this relationship persists after accounting for confidence magnitude and query difficulty. We first establish the basic association and then examine these two alternative explanations.

Experimental Setup.

We conduct the analysis on the full MATH500 benchmark using five models from three families: Qwen2.5-Math-1.5B and 7B, LLaMA3.1-8B-Instruct, and DeepSeek-R1-Distill-Qwen-1.5B and 7B. For each model, we use the final task-level prefix V obtained through confidence-only optimization. For every query X_i , we sample $K = 8$ Gaussian perturbations with $\sigma = 0.01$:

$$V^{(k)} = V + \epsilon^{(k)}, \quad \epsilon^{(k)} \sim \mathcal{N}(0, \sigma^2 I). \quad (18)$$

Under each perturbed prefix, the model greedily decodes a trajectory $Y_i^{(k)}$. Its trajectory-level confidence is defined as the length-normalized log-probability:

$$c_i^{(k)} = \frac{1}{T_i^{(k)}} \sum_{t=1}^{T_i^{(k)}} \log p_\theta \left(y_{i,t}^{(k)} \mid X_i, V^{(k)}, y_{i,<t}^{(k)} \right), \quad (19)$$

Model family	Prefix length	Batch size	Learning rate	Temp.	Top- p	Rep. penalty	Conf. tokens	Max. tokens
Qwen2.5-Math	20	16	1e-3	0.7	0.95	1.15	256	3,072
LLaMA3.1-Instruct	20	16	5e-6	0.7	0.95	1.05	256	3,072
DeepSeek-R1-Distill-Qwen	10	16	5e-4	0.6	0.95	1.00	1,024	8,192

Table 5: Model-specific optimization and generation settings. “Conf. tokens” denotes the maximum number of generated tokens used to compute the confidence objective.

Model	Spearman’s ρ	AUROC
Qwen2.5-Math-1.5B	-0.369	0.668
Qwen2.5-Math-7B	-0.403	0.687
LLaMA3.1-8B-Instruct	-0.413	0.683
DS-Distill-Qwen-1.5B	-0.308	0.709
DS-Distill-Qwen-7B	-0.359	0.745

Table 6: Association between local confidence variance and unperturbed correctness. Spearman’s ρ measures the correlation between $\mathcal{S}_i$ and correctness, while AUROC uses $-\mathcal{S}_i$ to distinguish correct from incorrect answers.

where $y_{i,t}^{(k)}$ denotes the t -th token in $Y_i^{(k)}$ and $T_i^{(k)}$ is the total sequence length. We quantify local confidence stability using the sample variance:

$$\mathcal{S}_i = \frac{1}{K-1} \sum_{k=1}^K \left(c_i^{(k)} - \bar{c}_i \right)^2, \quad \bar{c}_i = \frac{1}{K} \sum_{k=1}^K c_i^{(k)}, \quad (20)$$

where lower $\mathcal{S}_i$ indicates greater local stability, and $\bar{c}_i$ denotes the mean confidence across the perturbed trajectories.

For each query, we additionally greedily decode a reference trajectory under the unperturbed prefix V . We evaluate its final answer correctness using the standard benchmark evaluator and compute its original trajectory-level confidence $c_i^{(0)}$ following Eq. 19.

Accuracy across Stability Groups

We first examine whether local confidence stability is associated with correctness at the unperturbed prefix. For each query, we estimate $\mathcal{S}_i$ from perturbed trajectories and independently evaluate correctness using a trajectory decoded under the unperturbed prefix. We split queries at the model-specific median of $\mathcal{S}_i$ and report group accuracy with 95% bootstrap confidence intervals.

As shown in Figure 9, Low-Variance queries achieve higher unperturbed accuracy across all five models, with gaps ranging from 17.2 to 29.0 percentage points. This pattern holds for both general and reasoning-enhanced LLMs. Table 6 further shows consistently negative correlations between local variance and unperturbed correctness, with AUROCs ranging from 0.668 to 0.745. Importantly, stability is estimated from perturbed trajectories, whereas correctness is evaluated independently at the unperturbed prefix. The observed separation is therefore not merely induced by measuring variance and accuracy on the same rollouts. Instead, it suggests that neighborhood stability contains in-

Model	Gap	p_{perm}	ρ_{partial}	p_{partial}
Qwen2.5-Math-1.5B	5.4	0.086	-0.108	0.128
Qwen2.5-Math-7B	11.6	<0.001	-0.242	<0.001
LLaMA3.1-8B	23.3	<0.001	-0.283	<0.001
DS-Distill-Qwen-1.5B	30.5	<0.001	-0.560	<0.001
DS-Distill-Qwen-7B	8.7	<0.001	-0.297	<0.001

Table 7: Confidence-controlled association between local variance and perturbation accuracy. $\text{Gap}_{\text{strat}}$ is the Low-minus High-Variance accuracy after five-way stratification by mean confidence. Partial Spearman correlation controls for mean confidence continuously.

formation about the reliability of the optimized prefix itself across model families.

Controlling for Confidence Magnitude.

A remaining concern is that local variance may merely reflect confidence magnitude, as more confident queries may also be inherently more stable. To separate these effects, we divide queries into five equal-frequency strata according to their mean confidence across perturbed trajectories. Within each stratum, we split queries at the median of $\mathcal{S}_i$ and compare their perturbation accuracy, thereby reducing confidence differences between the Low- and High-Variance groups. We estimate confidence intervals using stratified bootstrap sampling and assess the accuracy gap with a stratified permutation test. We additionally compute the partial Spearman correlation between local variance and perturbation accuracy while controlling for mean confidence.

As shown in Table 7, Low-Variance queries achieve higher perturbation accuracy after confidence stratification across all five models. The differences are statistically significant on four models, with accuracy gaps ranging from 11.6 to 30.5 percentage points. Partial Spearman analysis yields the same pattern, showing significant negative correlations after controlling for mean confidence on these four models. These results suggest that local stability generally provides reliability information beyond confidence magnitude.

Qwen2.5-Math-1.5B is an exception, showing a non-significant 5.4-point gap and a near-zero partial correlation ($\rho = -0.108$, $p = 0.128$). Its local variance is strongly correlated with mean confidence ($\rho = -0.717$), suggesting that variance provides little additional information beyond confidence magnitude for this model.

Overall, these analyses show a consistent association between local confidence stability and answer accuracy across model families. The relationship persists among queries with

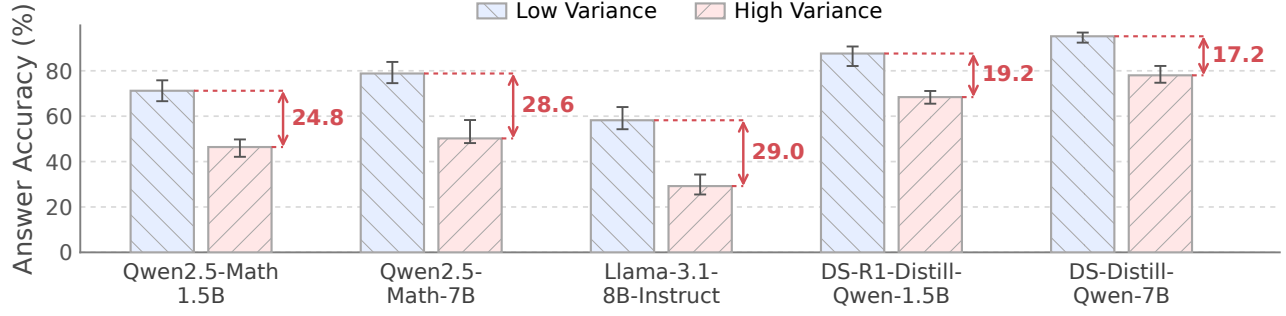


Figure 9: Answer accuracy under the unperturbed prefix for low- and high-variance query groups across five models. Groups are defined by the model-specific median of $\mathcal{S}_i$; error bars show 95% bootstrap confidence intervals, and red annotations indicate accuracy gaps.

comparable mean confidence and within different difficulty groups, indicating that it cannot be explained solely by confidence magnitude or easier queries. These findings provide empirical motivation for incorporating local stability into confidence-based test-time optimization. The main experiments further examine whether optimizing this signal improves reasoning performance.

C. Proofs and Additional Theoretical Analysis

The main paper presents the local-sensitivity results for Random Perturbation and Sharpness-Aware Perturbation. This section restates the quantities required by the proofs, makes the assumptions explicit, and provides the complete derivations. We distinguish behavioral confidence obtained after re-decoding from smooth confidence evaluated on a fixed trajectory, separating variation across decoded trajectories from local variation within a fixed decoding region.

Setup and Assumptions

We identify the prefix $V \in \mathbb{R}^{L \times d}$ with a vector in $\mathbb{R}^m$, where $m = Ld$. Under this vectorization, the Euclidean norm and inner product coincide with the Frobenius norm and inner product. For Random Perturbation, we draw K independent Gaussian perturbations:

$$V^{(k)} = V + \epsilon^{(k)}, \quad \epsilon^{(k)} \sim \mathcal{N}(0, \sigma^2 I). \quad (21)$$

Under each perturbed prefix $V^{(k)}$, the model greedily decodes a trajectory $Y^{(k)} = (y_1^{(k)}, \dots, y_{T_k}^{(k)})$. Following the notation in the main paper, its trajectory-level confidence is:

$$c^{(k)} = \frac{1}{T_k} \sum_{t=1}^{T_k} \log p_\theta(y_t^{(k)} | E'^{(k)}(X), y_{<t}^{(k)}), \quad (22)$$

where $E'^{(k)}(X) = [V^{(k)}; E(X)]$ denotes the input conditioned on $V^{(k)}$. Let $\bar{c} = K^{-1} \sum_{k=1}^K c^{(k)}$. The Random Perturbation objective uses the sample variance:

$$\mathcal{S}(V; X) = \frac{1}{K-1} \sum_{k=1}^K (c^{(k)} - \bar{c})^2. \quad (23)$$

For the population analysis, draw $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$. Let Z denote the greedy trajectory under $V + \epsilon$, and let $C = c(V + \epsilon)$

denote its trajectory-level confidence, defined analogously to Eq. 22. Thus, each $c^{(k)}$ is an independent realization of C . We assume that $\mathbb{E}[C^2] < \infty$.

For the smooth within-trajectory analysis, fix a trajectory $Y = (y_1, \dots, y_T)$ and define its teacher-forced confidence as:

$$f_Y(V) = \frac{1}{T} \sum_{t=1}^T \log p_\theta(y_t | E'(X), y_{<t}), \quad (24)$$

where $E'(X) = [V; E(X)]$ is the prefix-conditioned input.

We assume that V is an interior point of the decoding region associated with Y : there exists $r > 0$ such that every prefix $V + \delta$ satisfying $\|\delta\|_F \leq r$ greedily decodes Y . Let $g_Y = \nabla_V f_Y(V)$ and $H_Y = \nabla_V^2 f_Y(V)$.

We further assume that f_Y is sufficiently smooth around V so that, for $U \sim \mathcal{N}(0, I)$:

$$f_Y(V + \sigma U) = f_Y(V) + \sigma \langle g_Y, U \rangle_F + \frac{\sigma^2}{2} \langle U, H_Y U \rangle_F + R_\sigma(U), \quad (25)$$

where $\mathbb{E}[R_\sigma(U)^2] = \mathcal{O}(\sigma^6)$. This condition follows, for example, from a locally bounded third derivative together with an integrable Taylor remainder. As $\sigma \rightarrow 0$, the probability that a Gaussian perturbation leaves the fixed-decoding neighborhood decreases faster than any polynomial in σ . Under the same integrability condition, these tail events do not affect the polynomial orders derived below.

For Sharpness-Aware Perturbation, fix a trajectory Y within an update and let $\ell(V) = \mathcal{L}_{\text{conf}}(V; X, Y)$ denote the corresponding fixed-trajectory confidence loss. We assume that ℓ has a β -Lipschitz-continuous gradient on $\{V + \epsilon : \|\epsilon\|_F \leq \rho\}$. The stationary-point result additionally assumes that ℓ is twice continuously differentiable in a neighborhood of V .

Random Perturbation

Proposition 1 (Population behavioral sharpness). Let $C_1, \dots, C_K$ be the trajectory-level confidences induced by K independent Gaussian perturbations, and let $\mathcal{S}(V; X)$ be their sample variance in Eq. 23. Then:

$$\mathbb{E}[\mathcal{S}(V; X)] = \mathcal{S}_\sigma(V; X) = \text{Var}_{\epsilon \sim \mathcal{N}(0, \sigma^2 I)}[C]. \quad (26)$$

Moreover, the population sharpness satisfies:

$$\mathcal{S}_\sigma(V; X) = \mathbb{E}_Z[\text{Var}(C | Z)] + \text{Var}_Z(\mathbb{E}[C | Z]). \quad (27)$$

Proof. Let $\mu = \mathbb{E}[C]$ and $\bar{C} = K^{-1} \sum_{k=1}^K C_k$. The following identity holds:

$$\sum_{k=1}^K (C_k - \bar{C})^2 = \sum_{k=1}^K (C_k - \mu)^2 - K(\bar{C} - \mu)^2. \quad (28)$$

Taking expectations gives:

$$\begin{aligned} \mathbb{E} \left[\sum_{k=1}^K (C_k - \bar{C})^2 \right] &= K \text{Var}(C) - K \text{Var}(\bar{C}) \\ &= (K - 1) \text{Var}(C), \end{aligned} \quad (29)$$

where independence implies $\text{Var}(\bar{C}) = \text{Var}(C)/K$. Dividing by $K - 1$ proves Eq. 26.

For the second result, define $m(Z) = \mathbb{E}[C | Z]$. The centered confidence decomposes as:

$$C - \mathbb{E}[C] = (C - m(Z)) + (m(Z) - \mathbb{E}[C]). \quad (30)$$

The cross term has zero expectation because $\mathbb{E}[C - m(Z) | Z] = 0$. Taking the expectation of the squared identity yields Eq. 27. This argument treats Z as a discrete random variable and does not differentiate through decoding. $\square$

Corollary 1 (Control of large confidence deviations). For any $\tau > 0$, Chebyshev's inequality gives:

$$\Pr(|C - \mathbb{E}[C]| \geq \tau) \leq \frac{\mathcal{S}_\sigma(V; X)}{\tau^2}. \quad (31)$$

Thus, reducing population behavioral sharpness controls the probability of large confidence deviations under the perturbation distribution. This is a stability statement and does not by itself imply that the decoded answer is correct.

Corollary 2 (Contribution of behavioral changes). Let Y_0 be the trajectory decoded at the unperturbed prefix, and let $\phi(Z)$ denote a discrete behavioral descriptor of trajectory Z , such as its normalized final answer. Define $B = \mathbf{1}\{\phi(Z) \neq \phi(Y_0)\}$ and $p_{\text{chg}} = \Pr(B = 1)$. For $0 < p_{\text{chg}} < 1$, define the conditional means:

$$\mu_{\text{same}} = \mathbb{E}[C | B = 0], \quad \mu_{\text{chg}} = \mathbb{E}[C | B = 1]. \quad (32)$$

Because B is a function of Z , applying the law of total variance to $m(Z) = \mathbb{E}[C | Z]$ gives:

$$\text{Var}_Z(\mathbb{E}[C | Z]) \geq p_{\text{chg}}(1 - p_{\text{chg}})(\mu_{\text{chg}} - \mu_{\text{same}})^2. \quad (33)$$

The right-hand side is $\text{Var}_B(\mathbb{E}[C | B])$, which measures the confidence difference between the unchanged and changed groups. Thus, the bound is informative when behavioral changes are accompanied by a shift in mean confidence. If the two groups have identical mean confidence, it provides no information about how frequently behavior changes.

Proposition 2 (Smooth local sensitivity). Within a neighborhood where greedy decoding remains Y , the local confidence variance satisfies the following expansion as $\sigma \rightarrow 0$:

$$\text{Var}_\epsilon[f_Y(V + \epsilon)] = \sigma^2 \|\nabla_V f_Y(V)\|_F^2 + \mathcal{O}(\sigma^4). \quad (34)$$

If $\nabla_V f_Y(V) = 0$, the corresponding expansion is:

$$\text{Var}_\epsilon[f_Y(V + \epsilon)] = \frac{\sigma^4}{2} \|\nabla_V^2 f_Y(V)\|_F^2 + o(\sigma^4). \quad (35)$$

Proof. Write $\epsilon = \sigma U$, where $U \sim \mathcal{N}(0, I)$, and abbreviate $g = g_Y$ and $H = H_Y$. From Eq. 25, define:

$$A = \langle g, U \rangle_F, \quad Q = \frac{1}{2} \langle U, HU \rangle_F. \quad (36)$$

Then, $f_Y(V + \sigma U) = f_Y(V) + \sigma A + \sigma^2 Q + R_\sigma(U)$. The Gaussian moments satisfy:

$$\mathbb{E}[A] = 0, \quad \text{Var}(A) = \|g\|_F^2, \quad \text{Cov}(A, Q) = 0. \quad (37)$$

The last equality follows because every third-order centered Gaussian moment is zero. Moreover, $\text{Var}(\sigma^2 Q) = \mathcal{O}(\sigma^4)$. By the Cauchy-Schwarz inequality and $\mathbb{E}[R_\sigma(U)^2] = \mathcal{O}(\sigma^6)$, the remainder and its covariance terms contribute at most $\mathcal{O}(\sigma^4)$. Consequently, we obtain:

$$\text{Var}[f_Y(V + \sigma U)] = \sigma^2 \|g\|_F^2 + \mathcal{O}(\sigma^4). \quad (38)$$

This proves Eq. 34.

At a stationary point of f_Y , $g = 0$, and the quadratic term becomes dominant:

$$\text{Var}[f_Y(V + \sigma U)] = \sigma^4 \text{Var}(Q) + o(\sigma^4). \quad (39)$$

For a symmetric Hessian H and a standard Gaussian vector U , the following identities hold:

$$\mathbb{E}[U^\top H U] = \text{tr}(H), \quad \text{Var}(U^\top H U) = 2\|H\|_F^2. \quad (40)$$

Therefore, $\text{Var}(Q) = \frac{1}{2}\|H\|_F^2$, which proves Eq. 35. $\square$

This proposition characterizes the smooth within-trajectory component of behavioral sharpness; variation due to trajectory switching is captured by the between-trajectory term in Eq. 27.

Perturbation scale. The elementwise standard deviation σ corresponds to the following root-mean-square Frobenius radius:

$$\sqrt{\mathbb{E}[\|\epsilon\|_F^2]} = \sigma \sqrt{Ld}. \quad (41)$$

Thus, choosing $\sigma = \rho/\sqrt{Ld}$ would match the root-mean-square norm of the random perturbation to the SAP radius ρ . This relation aligns only their geometric scales and does not make the two objectives equivalent.

Sharpness-Aware Perturbation

Proposition 3 (First-order worst-case bound). Let the worst-case local increase be:

$$\Delta_\rho(V) = \max_{\|\epsilon\|_F \leq \rho} [\ell(V + \epsilon) - \ell(V)]. \quad (42)$$

Let $g = \nabla_V \ell(V)$. If ℓ has a β -Lipschitz-continuous gradient on the perturbation ball, the following bound holds:

$$|\Delta_\rho(V) - \rho \|g\|_F| \leq \frac{\beta \rho^2}{2}. \quad (43)$$

For $g \neq 0$, the gradient-guided perturbation $\epsilon^* = \rho g / \|g\|_F$ additionally satisfies:

$$|\ell(V + \epsilon^*) - \ell(V) - \rho \|g\|_F| \leq \frac{\beta \rho^2}{2}. \quad (44)$$

Accordingly, the implemented single-example SAP loss is $\mathcal{L}_{\text{SAP}} = \ell(V + \epsilon^*)$.

Proof. For any ϵ in the perturbation ball, the fundamental theorem of calculus gives:

$$\begin{aligned} \ell(V + \epsilon) - \ell(V) - \langle g, \epsilon \rangle_F \\ = \int_0^1 \langle \nabla_V \ell(V + t\epsilon) - g, \epsilon \rangle_F dt. \end{aligned} \quad (45)$$

Gradient Lipschitzness and Cauchy–Schwarz imply:

$$|\ell(V + \epsilon) - \ell(V) - \langle g, \epsilon \rangle_F| \leq \int_0^1 \beta t \|\epsilon\|_F^2 dt = \frac{\beta}{2} \|\epsilon\|_F^2. \quad (46)$$

For the upper bound, Eq. 46 gives:

$$\Delta_\rho(V) \leq \max_{\|\epsilon\|_F \leq \rho} \langle g, \epsilon \rangle_F + \frac{\beta \rho^2}{2} = \rho \|g\|_F + \frac{\beta \rho^2}{2}. \quad (47)$$

When $g \neq 0$, evaluating the inner objective at $\epsilon^* = \rho g / \|g\|_F$ gives:

$$\Delta_\rho(V) \geq \ell(V + \epsilon^*) - \ell(V) \geq \rho \|g\|_F - \frac{\beta \rho^2}{2}. \quad (48)$$

When $g = 0$, choosing $\epsilon = 0$ gives $\Delta_\rho(V) \geq 0$, while the upper bound gives $\Delta_\rho(V) \leq \beta \rho^2 / 2$. Hence, Eq. 43 holds in both cases. Finally, substituting ϵ^* into Eq. 46 proves Eq. 44. $\square$

Effect of stop-gradient. For $g \neq 0$, the bounds above concern the value of the perturbed objective and are unchanged by the stop-gradient operation. Treating ϵ^* as constant gives the outer gradient:

$$\nabla_V^{\text{sg}} \mathcal{L}_{\text{SAP}} = \nabla_V \ell(V + \epsilon^*). \quad (49)$$

Gradient Lipschitzness further gives:

$$\|\nabla_V^{\text{sg}} \mathcal{L}_{\text{SAP}} - g\|_F \leq \beta \rho. \quad (50)$$

Thus, stop-gradient removes derivatives through the construction of ϵ^* while retaining the gradient evaluated at the perturbed prefix.

Proposition 4 (Stationary-point curvature). Suppose that $\nabla_V \ell(V) = 0$ and that ℓ is twice continuously differentiable in a neighborhood of V . As $\rho \rightarrow 0$, the ideal inner maximization satisfies:

$$\Delta_\rho(V) = \frac{\rho^2}{2} [\lambda_{\max}(\nabla_V^2 \ell(V))]_+ + o(\rho^2), \quad (51)$$

where $[a]_+ = \max(a, 0)$.

Proof. Let $H = \nabla_V^2 \ell(V)$. Twice continuous differentiability and $\nabla_V \ell(V) = 0$ yield the following expansion, uniformly over $\|\epsilon\|_F \leq \rho$ as $\rho \rightarrow 0$:

$$\ell(V + \epsilon) - \ell(V) = \frac{1}{2} \langle \epsilon, H \epsilon \rangle_F + o(\|\epsilon\|_F^2). \quad (52)$$

Writing $\epsilon = \rho U$ with $\|U\|_F \leq 1$ gives:

$$\Delta_\rho(V) = \frac{\rho^2}{2} \max_{\|U\|_F \leq 1} \langle U, H U \rangle_F + o(\rho^2). \quad (53)$$

Algorithm 1: TASCO with Random Perturbation

Require: Frozen LLM M_θ , test set $\mathcal{D}_{\text{test}}$, warm-start prefix V_0 , optimizer $\mathcal{O}$, number of epochs E , perturbation scale σ , number of perturbations $K \geq 2$, and stability weight λ_{rand}

Ensure: Optimized prefix V

```

1: Initialize  $V \leftarrow V_0$ 
2: for  $e = 1, \dots, E$  do
3:   Randomly shuffle  $\mathcal{D}_{\text{test}}$  and partition it into mini-batches
4:   for each mini-batch  $\mathcal{B}$  do
5:     Generate trajectories  $\mathbf{Y}_{\mathcal{B}}$  under  $V$ 
6:      $\mathcal{L}_{\text{conf}} \leftarrow \mathcal{L}_{\mathcal{B}}(V; \mathbf{Y}_{\mathcal{B}})$ 
7:     for  $k = 1, \dots, K$  do
8:       Sample  $\epsilon^{(k)} \sim \mathcal{N}(0, \sigma^2 I)$  and set  $V^{(k)} \leftarrow V + \epsilon^{(k)}$ 
9:       Greedily generate trajectories  $\mathbf{Y}_{\mathcal{B}}^{(k)}$  under  $V^{(k)}$ 
10:      Compute  $\{c_X^{(k)} : X \in \mathcal{B}\}$  using Eq. 22
11:    end for
12:    Compute  $\mathcal{L}_{\text{rand}}$  from  $\{c_X^{(k)}\}$  using Eq. 23
13:     $g \leftarrow \nabla_V (\mathcal{L}_{\text{conf}} + \lambda_{\text{rand}} \mathcal{L}_{\text{rand}})$ 
14:     $V \leftarrow \mathcal{O}(V, g)$ 
15:  end for
16: end for
17: return  $V$ 

```

If $\lambda_{\max}(H) > 0$, the maximum is attained by a unit eigenvector associated with $\lambda_{\max}(H)$. If $\lambda_{\max}(H) \leq 0$, it is attained at $U = 0$. Therefore:

$$\max_{\|U\|_F \leq 1} \langle U, H U \rangle_F = [\lambda_{\max}(H)]_+. \quad (54)$$

This proves Eq. 51. $\square$

This proposition characterizes the ideal worst-case objective at stationarity, whereas Proposition 3 characterizes the implemented gradient-guided perturbation when $g \neq 0$.

Remark. Proposition 4 characterizes the ideal worst-case inner problem. At an exact stationary point, the first-order perturbation $\epsilon^* = \rho g / \|g\|_F$ is undefined and does not recover the leading Hessian eigenvector. The proposition therefore explains the local geometry of the robust objective rather than claiming that the implemented first-order perturbation solves the stationary second-order problem.

Relationship Between the Two Local Objectives

The two analyses reveal distinct views of local geometry. To compare them, suppose temporarily that both perturbation strategies are applied to the same smooth scalar function $h(V)$ and that decoding remains fixed. Away from stationary points, the random-perturbation variance satisfies:

$$\text{Var}_\epsilon[h(V + \epsilon)] = \sigma^2 \|\nabla_V h(V)\|_F^2 + \mathcal{O}(\sigma^4). \quad (55)$$

By contrast, the worst-case local increase satisfies:

$$\max_{\|\epsilon\|_F \leq \rho} [h(V + \epsilon) - h(V)] = \rho \|\nabla_V h(V)\|_F + \mathcal{O}(\rho^2). \quad (56)$$

Thus, the random objective aggregates sensitivity over isotropic directions, whereas the worst-case objective selects the most adverse direction.

At a stationary point with Hessian H_h , their leading curvature terms depend on $\|H_h\|_F^2$ and $[\lambda_{\max}(H_h)]_+$, respectively. Their relationship is bounded by:

$$[\lambda_{\max}(H_h)]_+ \leq \|H_h\|_2 \leq \|H_h\|_F. \quad (57)$$

This distinguishes the largest positive curvature from aggregate curvature across directions. In TASCO, Random Perturbation operates on re-decoded trajectory-level log-likelihood, whereas SAP operates on the fixed-trajectory entropy loss. Therefore, these relations clarify their different geometric interpretations but do not establish numerical equivalence or an ordering between the objectives.

Summary. Together, these analyses show how the two TASCO objectives control complementary forms of local confidence sensitivity. Random Perturbation regularizes distributional variation across perturbed behaviors, whereas SAP controls worst-case sensitivity along an adverse local direction. These results establish local confidence robustness under prefix perturbations rather than a guarantee of answer correctness. Section B separately examines the empirical relationship between local stability and answer accuracy.

D. Complete Optimization Algorithms

This section provides the complete optimization procedures for the two TASCO variants. Both algorithms optimize a task-level prefix V over the unlabeled test set while keeping the LLM parameters θ frozen. They differ only in how local confidence stability is estimated and incorporated into prefix optimization.

Random Perturbation. Algorithm 1 presents the Random Perturbation variant. At each update, it samples K Gaussian perturbations around the current prefix and generates a trajectory under each perturbed prefix. The resulting trajectory-level confidence variance is averaged across the mini-batch and combined with the confidence loss to update V .

Sharpness-Aware Perturbation. Algorithm 2 presents the Sharpness-Aware Perturbation variant. It first generates one trajectory per input under the current prefix and holds these trajectories fixed throughout the update. A gradient-guided perturbation is then constructed from the confidence loss, after which the prefix is updated using the loss evaluated at the perturbed prefix. The perturbation is treated as constant during the second backward pass.

Computational Cost. Random Perturbation captures richer distributional information but requires K independently decoded perturbed rollouts per input, introducing additional adaptation cost. Sharpness-Aware Perturbation avoids these rollouts through a single gradient-guided perturbation on the fixed trajectory, substantially reducing autoregressive decoding cost relative to Random Perturbation. The reported token-efficiency results characterize generation length after adaptation. Moreover, the learned prefix transfers to unseen datasets without re-optimization, avoiding additional adaptation cost in cross-distribution settings.

Algorithm 2: TASCO with Sharpness-Aware Perturbation

Require: Frozen LLM M_θ , test set $\mathcal{D}_{\text{test}}$, warm-start prefix V_0 , optimizer $\mathcal{O}$, number of epochs E , and perturbation radius ρ

Ensure: Optimized prefix V

- 1: Initialize $V \leftarrow V_0$
- 2: **for** $e = 1, \dots, E$ **do**
- 3: Randomly shuffle $\mathcal{D}_{\text{test}}$ and partition it into mini-batches
- 4: **for** each mini-batch $\mathcal{B}$ **do**
- 5: Generate trajectories $\mathbf{Y}_{\mathcal{B}}$ under V and hold them fixed
- 6: $g \leftarrow \nabla_V \mathcal{L}_{\mathcal{B}}(V; \mathbf{Y}_{\mathcal{B}})$
- 7: **if** $\|g\|_F > 0$ **then**
- 8: $\epsilon^* \leftarrow \rho g / \|g\|_F$
- 9: **else**
- 10: $\epsilon^* \leftarrow 0$
- 11: **end if**
- 12: $g_{\text{SAP}} \leftarrow \nabla_V \mathcal{L}_{\mathcal{B}}(V + \text{StopGrad}(\epsilon^*); \mathbf{Y}_{\mathcal{B}})$
- 13: $V \leftarrow \mathcal{O}(V, g_{\text{SAP}})$
- 14: **end for**
- 15: **end for**
- 16: **return** V

E. Further Analysis of Reasoning Effectiveness

This section examines TASCO beyond final-answer accuracy, focusing on whether its gains remain consistent across models and runs and are reflected in the reasoning process itself. We also assess its sensitivity to the perturbation scale.

Evaluation on Reasoning-Enhanced Models.

We first examine whether TASCO also benefits reasoning-enhanced LLMs, whose pretrained reasoning behavior differs substantially from that of general instruction-tuned models. We evaluate DeepSeek-R1-Distill-Qwen-1.5B and DeepSeek-R1-Distill-Qwen-7B on five mathematical reasoning benchmarks. In addition to the original model, we compare TASCO with test-time reasoning-control methods, including s1, CoD, and $\alpha 1$. We report answer accuracy and average generation length to jointly assess reasoning effectiveness and token efficiency.

As shown in Table 9, TASCO achieves the highest average accuracy at both model scales and consistently outperforms the base model across all five benchmarks. It improves average accuracy from 42.2% to 49.0% on DeepSeek-R1-Distill-Qwen-1.5B while reducing generation length by 14.1%, and from 56.1% to 61.1% on the 7B model with an 8.8% reduction. Although some reasoning-control baselines produce shorter outputs, their accuracy gains are smaller or inconsistent. TASCO thus offers a stronger balance between accuracy and generation efficiency, demonstrating that stability-aware confidence optimization also benefits models already trained for deliberative reasoning.

Evaluation Across Random Seeds

We additionally evaluate TASCO and selected baselines using three independent random seeds. To limit the cost of

Method	MATH500	AMC23	Minerva	AIME24	GPQA	Avg.
Qwen2.5-Math-1.5B						
Zero-Shot CoT	40.2 (± 1.8)	34.1 (± 2.4)	9.4 (± 1.4)	6.2 (± 3.5)	19.4 (± 2.1)	21.9 (± 0.5)
LTPO	57.6 (± 2.1)	47.2 (± 2.6)	12.4 (± 1.5)	8.8 (± 3.1)	21.5 (± 1.6)	28.6 (± 2.2)
TTSV	69.2 (± 1.6)	45.4 (± 2.0)	19.2 (± 1.4)	6.2 (± 2.6)	24.2 (± 0.6)	32.9 (± 1.4)
TASCO-RP	70.8 (± 0.4)	49.9 (± 0.9)	22.1 (± 1.3)	9.9 (± 1.0)	26.7 (± 1.1)	35.9 (± 0.5)
TASCO-SAP	71.6 (± 0.9)	49.4 (± 1.4)	20.1 (± 2.2)	11.8 (± 1.6)	26.2 (± 0.6)	36.4 (± 0.7)
Qwen2.5-Math-7B						
Zero-Shot CoT	51.8 (± 1.6)	42.9 (± 2.0)	14.1 (± 2.9)	13.3 (± 2.3)	30.7 (± 1.3)	30.8 (± 1.8)
LTPO	65.5 (± 2.2)	42.8 (± 3.1)	18.2 (± 2.6)	17.5 (± 2.8)	33.6 (± 2.5)	35.7 (± 2.8)
TTSV	70.0 (± 2.2)	57.8 (± 3.7)	32.8 (± 2.5)	17.0 (± 3.4)	36.0 (± 2.5)	42.7 (± 3.2)
TASCO-RP	74.5 (± 1.2)	66.7 (± 2.0)	38.1 (± 2.1)	20.4 (± 2.4)	37.7 (± 2.6)	47.2 (± 2.1)
TASCO-SAP	72.6 (± 0.8)	65.2 (± 2.0)	39.0 (± 2.4)	23.4 (± 2.4)	38.5 (± 2.8)	47.8 (± 1.8)
LLaMA3.1-8B-Instruct						
Zero-Shot CoT	47.0 (± 0.8)	25.1 (± 0.9)	21.7 (± 0.7)	7.9 (± 1.2)	28.7 (± 0.7)	26.1 (± 0.8)
LTPO	48.9 (± 0.8)	26.5 (± 1.4)	20.8 (± 2.8)	10.8 (± 1.4)	30.1 (± 0.8)	27.5 (± 0.9)
TTSV	47.1 (± 0.7)	22.5 (± 2.8)	19.6 (± 2.7)	5.2 (± 3.1)	27.2 (± 1.7)	25.0 (± 1.6)
TASCO-RP	47.2 (± 1.2)	28.6 (± 0.8)	25.3 (± 1.5)	10.9 (± 1.2)	32.7 (± 0.7)	29.3 (± 1.5)
TASCO-SAP	49.7 (± 0.8)	26.4 (± 2.0)	26.0 (± 0.7)	12.2 (± 1.2)	32.2 (± 0.8)	29.2 (± 0.8)

Table 8: Accuracy (%) across three independent random seeds. Each entry reports the mean accuracy followed by the standard deviation in gray. The average is computed across the five benchmarks for each run and then aggregated across seeds.

repeated test-time optimization, we include representative methods covering no adaptation, latent-variable optimization, and confidence-only prefix optimization: Zero-Shot CoT, LTPO, and TTSV. We evaluate both TASCO variants under the same protocol and report mean accuracy with standard deviation in Table 8. This evaluation provides more reliable performance estimates and quantifies variation across independent runs.

Semantic Evaluation of Reasoning Processes

Experimental Setup. We use DeepSeek-V4-Flash as an external judge to examine how TASCO affects intermediate reasoning. We compare paired trajectories generated by Confidence Only and TASCO-SAP using Qwen2.5-Math-7B, LLaMA3.1-8B-Instruct, and DeepSeek-R1-Distill-Qwen-7B on the full MATH500 and Minerva Math test sets, containing 500 and 272 examples, respectively. For each trajectory, the judge receives only the original problem and the generated reasoning process; no ground-truth answer is provided. Common explicit final-answer spans are masked to prevent the judge from inferring reasoning quality from answer correctness. We enable thinking mode with high reasoning effort and request a structured JSON annotation.

The judge first divides each trajectory into semantically meaningful steps. Each step receives a correctness label from {correct, incorrect, uncertain}, an informativeness label from {informative, redundant, off_track}, and a functional role. The judge also determines whether the trajectory commits prematurely to an answer or reasoning direction before sufficient support has been established. All reported metrics are computed from

these annotations rather than directly assigned by the judge.

External-Judge Prompt. The prompt template used for reasoning-process annotation is shown below. The placeholders are replaced with the original problem and the corresponding trajectory after final-answer masking.

External-Judge Prompt. We use the following condensed prompt to summarize the instructions given to the external judge. The complete prompt and JSON schema are provided in the released implementation.

Reasoning-Process Evaluation Prompt

System: You are an expert evaluator of mathematical reasoning. Evaluate the reasoning process independently of its final answer, writing style, verbosity, or expressed confidence. Divide the trajectory into semantically meaningful steps. Label each step by:

- correctness: correct, incorrect, or uncertain;
- informativeness: informative, redundant, or off_track.

Determine whether the trajectory commits prematurely to an answer or reasoning direction before sufficient support is established. Return all annotations as a valid JSON object.

User: Evaluate the following reasoning trajectory.

Problem: <Problem>

Reasoning trajectory: <Reasoning Trajectory>

Evaluation Metrics. For trajectory i with J_i semantic steps, let c_{ij} and u_{ij} denote the correctness and informativeness labels of step j , respectively. Let E_i denote the subset

Method	AIME24		AIME25		MATH500		AMC23		Minerva		Average	
	Acc.	Tok.	Acc.	Tok.	Acc.	Tok.	Acc.	Tok.	Acc.	Tok.	Acc.	Tok.
DeepSeek-R1-Distill-Qwen-1.5B												
Base	22.8	7580	22.5	7674	78.6	3700	59.4	6242	27.6	5057	42.2	6051
s1	25.8	7778	<u>25.4</u>	7982	80.2	4623	61.9	6428	29.4	6127	44.5 $\uparrow$ 2.4	6588 $\uparrow$ 8.9%
CoD	26.7	7196	20.8	7552	79.4	3267	<u>66.2</u>	5293	32.0	4041	<u>45.0</u> $\uparrow$ 2.8	5470 $\downarrow$ 9.6%
α 1	30.8	<u>6841</u>	12.9	6529	81.0	<u>3541</u>	63.1	<u>4310</u>	<u>33.1</u>	4589	44.2 $\uparrow$ 2.0	5162 $\downarrow$ 14.7%
TASCO-SAP	<u>30.4</u>	6751	29.2	<u>6838</u>	81.4	3616	69.4	4282	34.6	<u>4507</u>	49.0 $\uparrow$ 6.8	<u>5199</u> $\downarrow$ 14.1%
DeepSeek-R1-Distill-Qwen-7B												
Base	44.2	6899	25.0	7099	89.4	3399	79.4	4672	<u>42.6</u>	4416	56.1	5297
s1	37.5	7368	24.2	7041	89.0	4072	78.1	5271	40.4	4986	53.8 $\downarrow$ 2.3	5748 $\uparrow$ 8.5%
CoD	<u>45.0</u>	<u>6301</u>	<u>33.3</u>	6582	88.0	2071	85.6	3645	40.1	2440	<u>58.4</u> $\uparrow$ 2.3	4208 $\downarrow$ 20.6%
α 1	36.7	6831	32.5	6638	89.8	3848	90.6	4529	<u>42.6</u>	4223	<u>58.4</u> $\uparrow$ 2.3	5214 $\downarrow$ 1.6%
TASCO-SAP	49.2	6188	34.2	6652	90.4	<u>3041</u>	<u>88.1</u>	<u>4287</u>	43.4	<u>3988</u>	61.1 $\uparrow$ 4.9	<u>4831</u> $\downarrow$ 8.8%

Table 9: Accuracy (%) and average generation length across five reasoning benchmarks. Accuracy is reported to one decimal place, and token counts are rounded to the nearest integer. Average results are computed across the five datasets. Arrows denote absolute accuracy changes and relative token changes compared with Base. Best results are in **bold**, and second-best results are underlined.

Model	Method	MATH500				Minerva Math			
		SC $\uparrow$	SI $\uparrow$	PC $\downarrow$	RR $\downarrow$	SC $\uparrow$	SI $\uparrow$	PC $\downarrow$	RR $\downarrow$
Qwen2.5-Math-7B	Confidence Only	82.7	77.6	18.8	15.1	75.4	75.7	22.8	11.2
	TASCO-SAP	85.2	80.4	17.2	11.6	77.5	80.5	19.5	10.1
LLaMA3.1-8B-Instruct	Confidence Only	65.6	66.2	43.0	10.2	51.8	58.8	51.8	12.8
	TASCO-SAP	67.6	67.6	41.4	8.8	63.7	69.6	40.4	10.2
DS-Distill-Qwen-7B	Confidence Only	91.2	83.6	6.8	11.6	69.5	69.7	31.3	11.9
	TASCO-SAP	94.2	85.1	4.8	11.9	72.1	72.6	29.3	10.6

Table 10: Reasoning-process evaluation using DeepSeek-V4-Flash as the external judge. All values are percentages.

of steps labeled either correct or incorrect. Step Correctness is defined as:

$$SC_i = \frac{\sum_{j \in E_i} \mathbf{1}\{c_{ij} = \text{correct}\}}{|E_i|}, \quad (58)$$

where $\mathbf{1}\{\cdot\}$ is the indicator function and uncertain steps are excluded. Higher SC indicates that a larger proportion of assessable steps are mathematically or logically valid.

Step Informativeness measures the proportion of steps that introduce useful information and advance the solution:

$$SI_i = \frac{1}{J_i} \sum_{j=1}^{J_i} \mathbf{1}\{u_{ij} = \text{informative}\}. \quad (59)$$

Higher SI indicates more productive reasoning.

Redundancy Rate measures the proportion of steps that mainly repeat or restate previously established information:

$$RR_i = \frac{1}{J_i} \sum_{j=1}^{J_i} \mathbf{1}\{u_{ij} = \text{redundant}\}. \quad (60)$$

Lower RR indicates less repetitive reasoning.

Let p_i denote the trajectory-level premature-commitment judgment. We define the corresponding indicator and its aggregate rate over N trajectories as:

$$PC_i = \mathbf{1}\{p_i = \text{true}\}, \quad PC = \frac{1}{N} \sum_{i=1}^N PC_i. \quad (61)$$

Lower PC indicates that the model is less likely to commit prematurely to an insufficiently supported reasoning direction. We average SC, SI, and RR across trajectories and report all four metrics as percentages.

Sensitivity to Perturbation Scale. We further examine the sensitivity of TASCO to the Gaussian scale σ and perturbation radius ρ on four additional benchmarks. As shown in Fig. 10, both variants outperform confidence-only optimization across a broad range of perturbation scales. The default settings, $\sigma = 0.01$ and $\rho = 0.8$, achieve the best or near-best accuracy across datasets, while moderate changes generally produce comparable results. These findings indicate that TASCO is not sensitive to a narrowly tuned perturbation scale and that the selected settings generalize well across benchmarks.

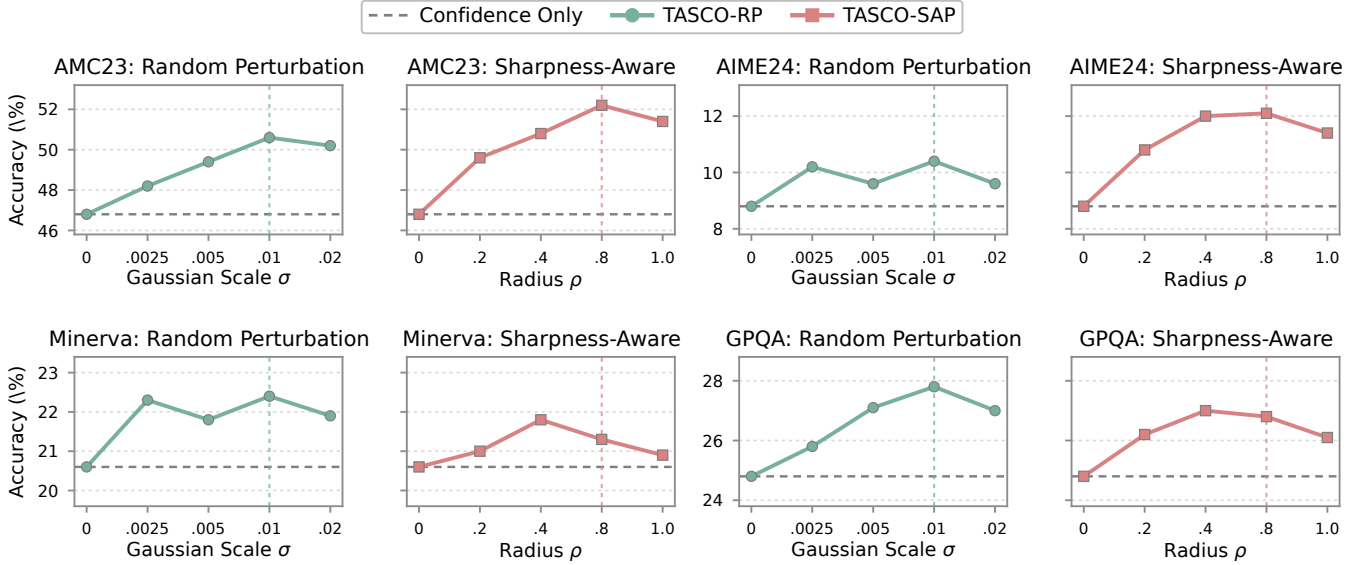


Figure 10: Sensitivity of TASCO to perturbation scales on four additional benchmarks using Qwen2.5-Math-1.5B. Each benchmark contains separate panels for Random Perturbation and Sharpness-Aware Perturbation.

Results. As shown in Table 10, TASCO consistently improves all four reasoning-process metrics across both datasets. It produces a larger proportion of correct and informative steps while reducing redundant reasoning and premature commitment. These results suggest that stability-aware optimization affects not only final-answer accuracy but also how the model develops and maintains its reasoning process.

F. Case Studies

This section presents case studies that illustrate the experimental prompting protocol and compare the reasoning behaviors induced by Confidence Only and TASCO.

Prompt and Example. We first provide an AIME24 example illustrating the complete prompt and response format. As shown in Table 11, the system message instructs the model to reason step by step and place its final answer within `\boxed{ }`, while the original problem is provided as the user message. TASCO adds no textual instructions; its learned continuous prefix is prepended only at the embedding level. The table reports the complete response and the final answer extracted by the benchmark parser. Subsequent case studies use the same prompt format and omit these repeated instructions.

Reasoning Patterns. Table 12 reveals a recurring contrast between the two methods. Confidence Only often commits early to a locally plausible solution and continues confidently after discarding a necessary condition. Across the three cases, it cancels a potentially zero factor, confuses simultaneous constraints with their union, or violates a coprimality requirement. TASCO instead preserves these constraints throughout the derivation and retains valid reasoning branches until they are properly resolved. These examples suggest that stability-aware optimization mitigates premature commitment to frag-

ile reasoning paths and promotes more consistent and reliable multi-step reasoning.

SYSTEM: Please reason step by step, and put your final answer within `\boxed{ }`.

USER: $[\mathbf{v}_1][\mathbf{v}_2][\mathbf{v}_3] \cdots [\mathbf{v}_{20}]$ Every morning Aya goes for a 9-kilometer-long walk and stops at a coffee shop afterwards. When she walks at a constant speed of s kilometers per hour, the walk takes her 4 hours, including t minutes spent in the coffee shop. When she walks at $s + 2$ kilometers per hour, the walk takes her 2 hours and 24 minutes, including t minutes spent in the coffee shop. Suppose Aya walks at $s + \frac{1}{2}$ kilometers per hour. Find the number of minutes the walk takes her, including the t minutes spent in the coffee shop.

GROUND TRUTH: 204

TASCO REASONING:

To solve this problem, we first determine the values of s and t using the two given walking times.

Step 1: Formulate the first time constraint. When Aya walks at s kilometers per hour, the walking time is $\frac{9}{s}$ hours, while the time spent in the coffee shop is $\frac{t}{60}$ hours. Since the total time is 4 hours,

$$\frac{9}{s} + \frac{t}{60} = 4.$$

Step 2: Formulate the second time constraint. When Aya walks at $s + 2$ kilometers per hour, the total duration is 2 hours and 24 minutes, or 2.4 hours. Therefore,

$$\frac{9}{s+2} + \frac{t}{60} = 2.4.$$

Step 3: Solve for s and t . Solving the two equations gives

$$(s, t) = (-3, 240) \quad \text{or} \quad (s, t) = (2.5, 24).$$

Because a speed of -3 kilometers per hour is not physically meaningful, we discard the first solution. Hence,

$$s = 2.5, \quad t = 24.$$

Step 4: Compute the requested duration. At the new speed,

$$s + \frac{1}{2} = 2.5 + 0.5 = 3$$

kilometers per hour. Walking 9 kilometers therefore takes

$$\frac{9}{3} = 3$$

hours. Including the 24 minutes spent in the coffee shop, the total duration is

$$3 \times 60 + 24 = 204$$

minutes.

Therefore, the walk takes $\boxed{204}$ minutes.

Table 11: Prompt and TASCO reasoning example on AIME24. The vectors shown before the user question schematically represent the learned continuous prefix prepended at the embedding level rather than textual tokens.

CASE 1: POLYNOMIAL ROOT FINDING

QUESTION: Find the roots of

$$(x - 3)^3 + (x - 7)^3 = (2x - 10)^3.$$

GROUND TRUTH: 3, 5, 7

CONFIDENCE-ONLY REASONING (INCORRECT):

Step 1: To solve the equation, we use the identity

$$a^3 + b^3 = (a + b)(a^2 - ab + b^2).$$

Step 2: Let $a = x - 3$ and $b = x - 7$. Applying the identity gives

$$(2x - 10)((x - 3)^2 - (x - 3)(x - 7) + (x - 7)^2) = (2x - 10)^3.$$

Step 3: Dividing both sides by $2x - 10$, we obtain

$$(x - 3)^2 - (x - 3)(x - 7) + (x - 7)^2 = (2x - 10)^2.$$

Step 4: Expanding both sides gives

$$x^2 - 6x + 9 - (x^2 - 10x + 21) + x^2 - 14x + 49 = 4x^2 - 40x + 100.$$

Simplifying yields

$$x^2 - 10x + 37 = 4x^2 - 40x + 100.$$

Step 5: Rearranging and factoring gives

$$3x^2 - 30x + 63 = 0, \quad x^2 - 10x + 21 = (x - 3)(x - 7) = 0.$$

Step 6: Therefore, the roots are $x = 3$ and $x = 7$, and the final answer is $\{3, 7\}$.

TASCO REASONING (CORRECT):

Step 1: Let $y = x - 5$. Then

$$x - 3 = y + 2, \quad x - 7 = y - 2, \quad 2x - 10 = 2y.$$

Step 2: The equation becomes

$$(y + 2)^3 + (y - 2)^3 = (2y)^3.$$

Expanding both sides gives

$$(y^3 + 6y^2 + 12y + 8) + (y^3 - 6y^2 + 12y - 8) = 8y^3.$$

Step 3: Combining like terms, rearranging, and factoring gives

$$2y^3 + 24y = 8y^3, \quad 6y(y^2 - 4) = 0.$$

Step 4: Thus,

$$y = 0, \quad y = 2, \quad y = -2.$$

Substituting $x = y + 5$ gives

$$x = 5, \quad x = 7, \quad x = 3.$$

Step 5: Therefore, the complete set of roots is $\{3, 5, 7\}$.

CASE 2: DOMAIN CONSTRAINTS

QUESTION: For which real values of x is the following function undefined?

$$f(x) = \frac{1}{|x^2 + 3x - 4| + |x^2 + 9x + 20|}.$$

GROUND TRUTH: -4

CONFIDENCE-ONLY REASONING (INCORRECT):

Step 1: To determine the values outside the domain, we find where the denominator is zero. Since both terms are absolute values,

$$|x^2 + 3x - 4| + |x^2 + 9x + 20| = 0$$

if and only if both quadratic expressions are zero.

Step 2: The first quadratic factors as

$$x^2 + 3x - 4 = (x + 4)(x - 1) = 0,$$

giving $x = -4$ or $x = 1$.

Step 3: The second quadratic factors as

$$x^2 + 9x + 20 = (x + 4)(x + 5) = 0,$$

giving $x = -4$ or $x = -5$.

Step 4: The roots of the two equations are combined by union rather than intersection. The reasoning therefore concludes that the denominator is zero at $x = -5, -4, 1$, giving $\{-5, -4, 1\}$.

TASCO REASONING (CORRECT):

Step 1: The denominator is a sum of two nonnegative terms, so it is zero only when both quadratic expressions are zero simultaneously.

Step 2: Solving the first equation gives

$$x^2 + 3x - 4 = (x + 4)(x - 1) = 0,$$

whose roots are $x = -4$ and $x = 1$.

Step 3: Solving the second equation gives

$$x^2 + 9x + 20 = (x + 4)(x + 5) = 0,$$

whose roots are $x = -4$ and $x = -5$.

Step 4: Because the two conditions must hold simultaneously, we take the intersection:

$$\{-4, 1\} \cap \{-4, -5\} = \{-4\}.$$

Therefore, the denominator is zero only at $x = -4$, and the final answer is $\{-4\}$.

CASE 3: COPRIMALITY CONSTRAINTS

QUESTION: If a and b are positive integers such that $\gcd(a, b) = 210$, $\text{lcm}(a, b) = 210^3$, and $a < b$, how many possible values are there for a ?

GROUND TRUTH: 8

CONFIDENCE-ONLY REASONING (INCORRECT):

Step 1: Using

$$\gcd(a, b) \text{lcm}(a, b) = ab,$$

we obtain

$$ab = 210 \cdot 210^3 = 210^4.$$

Step 2: Since $\gcd(a, b) = 210$, write

$$a = 210m, \quad b = 210n,$$

where $\gcd(m, n) = 1$. Substituting gives

$$mn = 210^2.$$

Step 3: The prime factorization is

$$210^2 = 2^2 \cdot 3^2 \cdot 5^2 \cdot 7^2.$$

Therefore, the total number of divisors of 210^2 is

$$(2 + 1)^4 = 81.$$

Step 4: Each divisor is treated as producing a coprime factor pair, even though many divisor pairs share prime factors. After removing symmetric pairs, the number of possible values of a is computed as

$$\frac{81 - 1}{2} = 40.$$

Therefore, the final answer is $\{40\}$.

TASCO REASONING (CORRECT):

Step 1: Using the relationship between the greatest common divisor and least common multiple gives

$$ab = \gcd(a, b) \operatorname{lcm}(a, b) = 210 \cdot 210^3 = 210^4.$$

Step 2: Write

$$a = 210m, \quad b = 210n,$$

where $\gcd(m, n) = 1$. Then

$$(210m)(210n) = 210^4, \quad mn = 210^2.$$

Step 3: Since

$$210^2 = 2^2 \cdot 3^2 \cdot 5^2 \cdot 7^2$$

and m and n are coprime, each complete prime power must be assigned entirely to either m or n . Splitting a prime power between the two factors would violate $\gcd(m, n) = 1$.

Step 4: There are four distinct prime powers, each of which can be assigned independently to either m or n . Hence, there are

$$2^4 = 16$$

ordered assignments. Since $a < b$, equivalently $m < n$, each unordered pair contributes exactly one valid value of a . Therefore,

$$\frac{16}{2} = 8.$$

The final answer is $\{8\}$.

Table 12: Three comparative case studies on polynomial roots, domain constraints, and coprime factorization. In each case, Confidence Only drops a condition identified earlier in the derivation, whereas TASCO preserves the condition and obtains the correct answer.